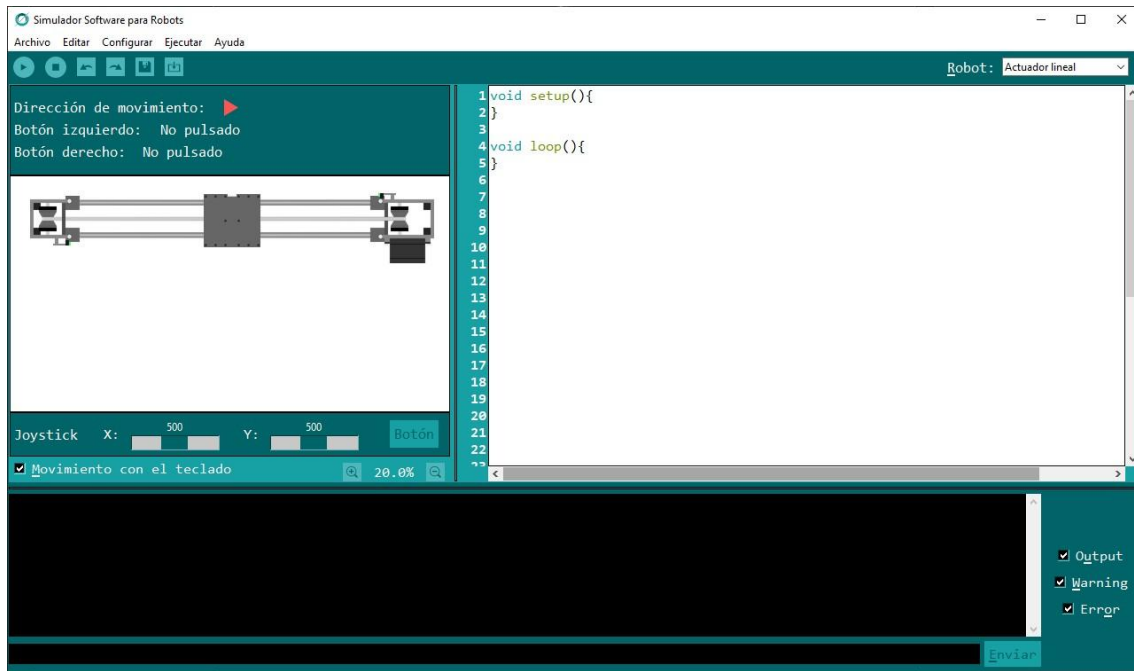


Simulador S4R



Práctica 4 – Simulador (v3 julio 2026)

Software para robots

Cristian González García gonzalezcristian@uniovi.es

Simulador desarrollado por Diego Fernández Suárez (2022), María Suárez Hevia (2023) y Nicolás Guerbartchouk Pérez (2026) y dirigido por Cristian González García

Índice

1.	Introducción.....	2
2.	Instalación.....	2
3.	Ejecución.....	3
4.	Manual de usuario	4
	Visión general del sistema	4
	Menú.....	5
	Barra de herramientas	9
	Representación gráfica de los robots.....	11
	Editor de código	20
	Consola de entrada/salida	20
	Creación de circuito	21
	Otras características para tener en cuenta	23
	Simulador 3D	24
5.	Manual de Calibración del Braccio físico	29
	Dónde vive la calibración y cuándo se aplica.....	29
	Material necesario	30
	El sketch de medición braccio_medicion_real.ino	31
	Procedimiento de medición, articulación por articulación	34
	Volcado de las medidas a la tabla de calibración	35
	Verificación de la calibración	35

1. Introducción

Este simulador fue desarrollado por el alumno Diego Fernández Suárez (2022), María Suárez Hevia (2023) y Nicolás Guerbartchouk Pérez (2026) de la Universidad de Oviedo para la asignatura de Software para Robots del Grado en ingeniería Informática del Software a través de la realización de su trabajo final de grado (TFG). Este fue dirigido por Cristian González García (gonzalezcristian@uniovi.es).

El manual es un extracto de dicho TFG, pero incluyendo mejoras y cambios realizados.

2. Instalación

El sistema deberá instalarse en un ordenador que tenga como sistema operativo Windows. Sería preferible que fuese Windows 10, ya que sobre este se ha probado la correcta ejecución del sistema. No obstante, el sistema debería funcionar correctamente instalándolo en cualquier versión posterior de Windows (Windows 11 en adelante) y en cualquier versión de Windows que cuente con soporte técnico hoy en día.

Ahora, se mostrarán los pasos necesarios para instalar el sistema de forma correcta:

1. Descargar archivo comprimido (.zip) de <https://github.com/gonzalezgarcia cristian/simulador-robotic-software> Este contiene los archivos necesarios para el correcto funcionamiento del ejecutable, además del propio ejecutable (.exe). Al descargarlo, deberíamos tener un archivo cuyo nombre será del tipo “simuladorSR-win”.
2. Descomprimir el archivo (en el directorio que queramos almacenar el programa y sin falta de descomprimir en un nuevo directorio, el archivo comprimido ya contiene un directorio que contiene el sistema). Deberíamos tener una situación similar a la de la Figura 1.

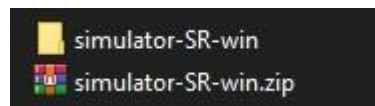


Figura 1 Directorio del Sistema – Descomprimido

3. Para ejecutar el programa principal accedemos a la carpeta descomprimida, en la que vemos lo mostrado en la Figura 2.

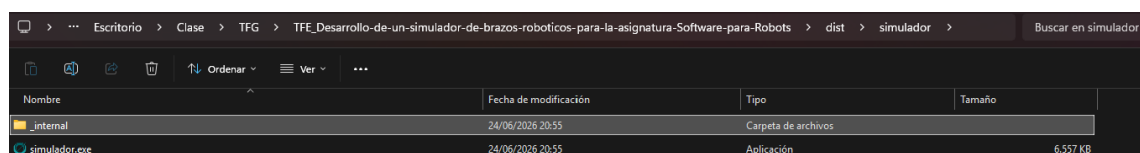


Figura 2 Directorio que contiene el sistema

El archivo que necesitamos ejecutar es el «main.exe», presente en la Figura 2. Si lo ejecutamos, ya podremos ver el sistema en funcionamiento.

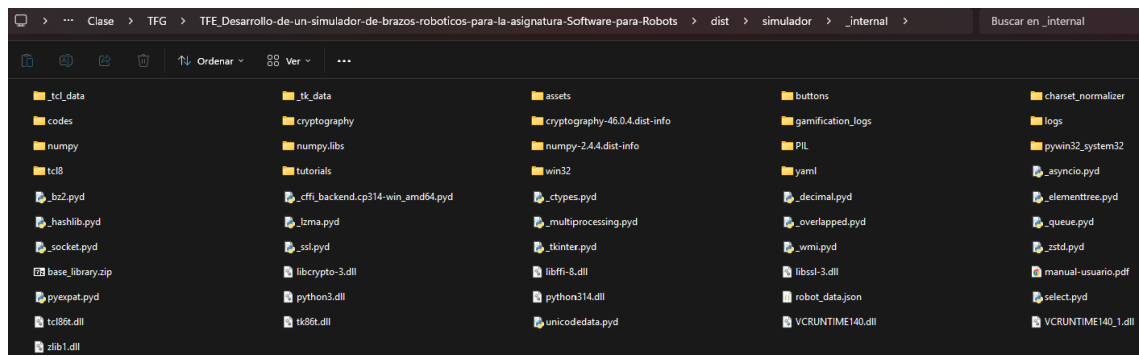


Figura 2.5 Carpeta de archivos de la aplicación

A continuación, se describe el resto del contenido de la carpeta `_internal`:

- Archivos de extensión “pyd” y “dll”: son bibliotecas del programa y en el archivo comprimido `base_library.zip` están los archivos de *bytecode* compilado.
- «`manual-usuario.pdf`»: es el manual de usuario que se ejecutará en el momento en que el usuario pulse sobre la opción “Manual de ayuda” del menú “Ayuda”.
- “`robot_data.json`”: contiene los datos necesarios para configurar los circuitos y los robots por defecto. Si se quiere cambiar los pines por defecto, se puede hacer en este archivo. No se recomienda hacer esto.
- Carpetas “`assets`” y “`buttons`”: contienen las imágenes png usadas para el *frontend* de la aplicación, en concreto “`buttons`” contiene los botones de la barra de herramientas y “`assets`” los gráficos de los robots y el icono de la aplicación.
- Carpeta «`logs`» y «`temp`»: se generan posteriormente. La primera almacena los logs que tendréis que enviar al profesor si este os los pide. La segunda contendrá el archivo de código compilado y las correspondientes las librerías usadas por el sistema.

3. Ejecución

Simplemente basta con entrar en la carpeta dónde se descomprimió el simulador previamente y ejecutar el archivo «`main.exe`» mostrado en la Figura 3.

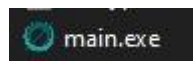


Figura 3 Main.exe

Una vez ejecutado, nos saldrán dos ventanas, la consola (que mostrará salida de Python si es que la hay, incluidos errores); y la ventana principal, que será la que nos permitirá utilizar toda la funcionalidad de la aplicación. Las ventanas se muestran en la Figura 4, la primera en la parte superior izquierda tapada por la segunda.

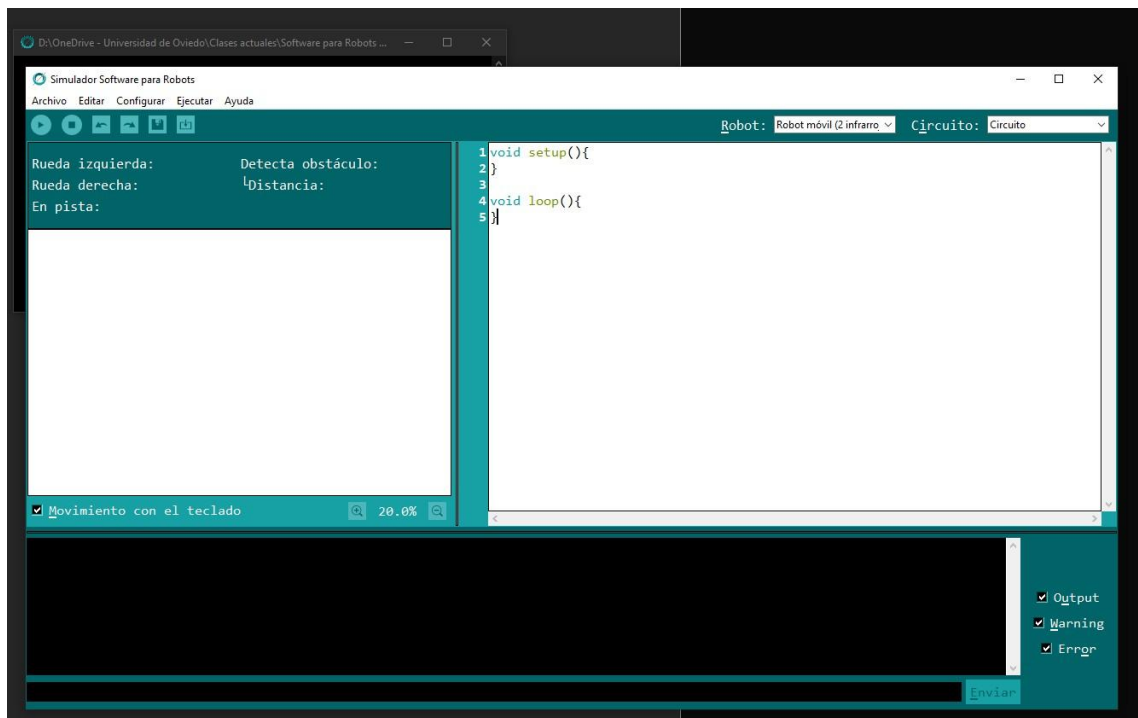


Figura 4 Simulador abierto

Para cerrar el sistema correctamente hay dos opciones:

- Cerrar desde el propio botón de cerrar de la ventana de la aplicación.
- Cerrar desde la opción dada en el menú Archivo de la aplicación.

No es recomendable cerrar la aplicación cerrando la consola o usando el administrador de tareas, es mejor cualquiera de las opciones propuestas anteriormente. Por ello, es mejor dejar estas opciones como las últimas en caso de que fallen las anteriores.

4. Manual de usuario

A continuación, se recoge el manual del usuario. Este manual se ha creado tomando como base el manual ya existente del simulador S4R, incluyendo la parte correspondiente al nuevo módulo de simulación 3D del brazo Braccio. Este manual será incluido dentro de la aplicación como ayuda para los usuarios, tratando de explicar las funcionalidades de la manera más simple posible.

Visión general del sistema

Al abrir la aplicación nos encontraremos con la ventana que contiene toda la funcionalidad del sistema, mostrada en la figura siguiente.

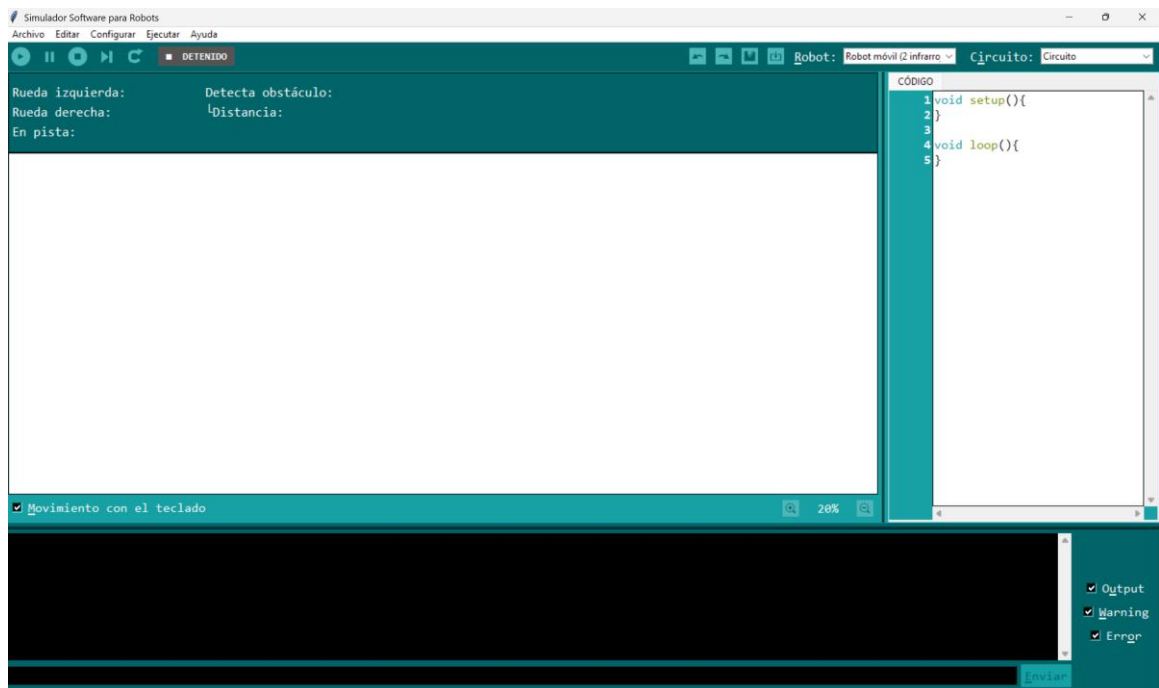


Figura 5 Pantalla principal del sistema en arranque por defecto

En ella podemos ver varios elementos:

- Arriba del todo encontramos el menú de la aplicación.
- Debajo justo del menú, encontramos la barra de herramientas, que contiene todas las herramientas básicas para ejecución, edición y apertura de sketches.
- Después, la interfaz se divide en tres componentes principales: a la izquierda la representación gráfica de los robots (2D para los robots heredados, 3D para el Braccio); a la derecha el editor de código; y abajo, la consola de entrada/salida.

Se explicarán en los siguientes apartados las funcionalidades que se podrán encontrar en cada componente mencionado.

Sobre esta estructura heredada, el módulo 3D del brazo Braccio introducido en este TFG añade tres elementos visibles desde el primer arranque: un indicador de estado de ejecución en la barra de herramientas que muestra en todo momento si el simulador está “■ DETENIDO”, “▶ EJECUTANDO” o “|| PAUSADO”; la reorganización dinámica del área central, que conmuta al renderizado 3D y muestra dos paneles laterales adicionales cuando el robot activo es el Braccio; y dos atajos de teclado nuevos —F8 para pausar/reanudar y F10 para avanzar paso a paso—. Estos elementos se describen en detalle en los apartados 9.3.3 y 9.3.9.

Menú

Comenzamos describiendo la funcionalidad del menú de la aplicación. En la siguiente figura se pueden ver las opciones ofrecidas por dicho menú.

Archivo Editar Configurar Ejecutar Ayuda

Figura 6 Menú de la aplicación

La primera opción es archivo, si pulsamos sobre ella, veremos que surge el menú desplegable siguiente.

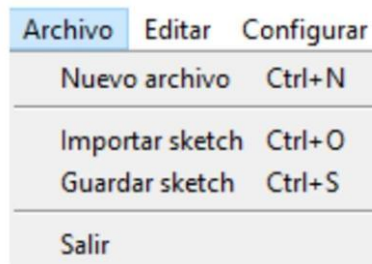


Figura 7 Menú archivo

En este menú podemos ver cuatro opciones:

- **Nuevo archivo:** nos permitirá crear la estructura de un nuevo sketch. Antes de crear el nuevo archivo nos saldrá un diálogo como el de la siguiente imagen. Si queremos crear el archivo escogemos sí, en otro caso basta con escoger no o cerrar el diálogo.

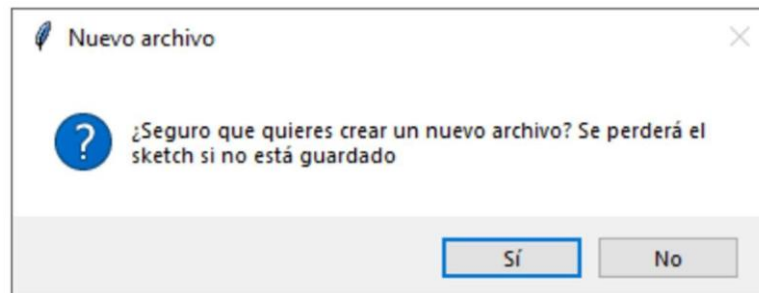


Figura 8 Diálogo de confirmación – Crear nuevo archivo

- **Importar sketch:** nos permitirá abrir un sketch que queramos editar. Emplea el diálogo típico de apertura de archivos de otras aplicaciones parecidas.
- **Guardar sketch:** nos permitirá guardar un sketch que hayamos creado/editado. Usa el diálogo clásico de guardado, con lo que podemos sobrescribir el archivo o guardar en el archivo que nosotros escojamos.
- **Salir:** permite salir de la aplicación, al igual que el botón cerrar de la ventana, solo que, en este caso, lanza un diálogo de confirmación como el de la siguiente imagen.

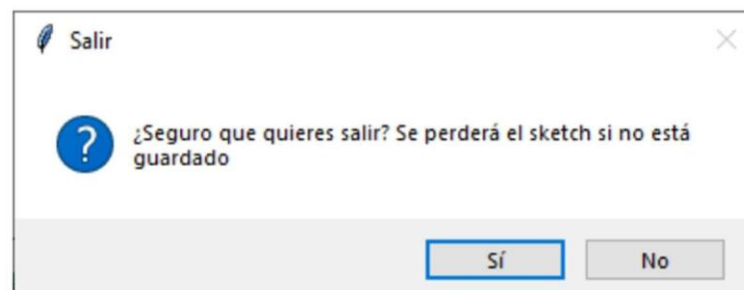


Figura 9 Diálogo de confirmación – Salir de la aplicación

La segunda opción del menú editar, si pulsamos sobre ella, veremos el menú mostrado en la siguiente imagen.

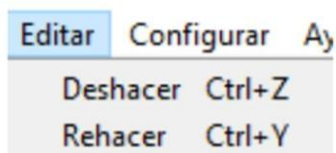


Figura 10 Menú editar

En este menú vemos dos opciones, que son deshacer y rehacer. Dichas opciones implican el que se deshaga o rehaga acciones del editor de código, sean editar, borrar, añadir o cualquier otra acción que tenga que ver con el sketch. Esto implica incluso que se puede deshacer el crear archivo o importar uno nuevo.

La siguiente opción es el menú de configuración, que contiene una sola opción, Configurar pines, cuyo mnemónico es 'ctrl + p'. Si accedemos a dicha opción, abriremos la siguiente ventana:

Figura 11 Ventana de configuración de pines del robot

La ventana que se muestra en la imagen anterior varía según el robot que esté elegido en la aplicación. La ventana mostrada en la figura anterior es para el robot móvil con 2 sensores de infrarrojos. Mientras que las figuras siguientes muestran las otras dos opciones, la primera el robot móvil de 4 infrarrojos y la segunda el actuador lineal.

Figura 12 Ventana de configuración – Robot móvil de 4 sensores infrarrojos

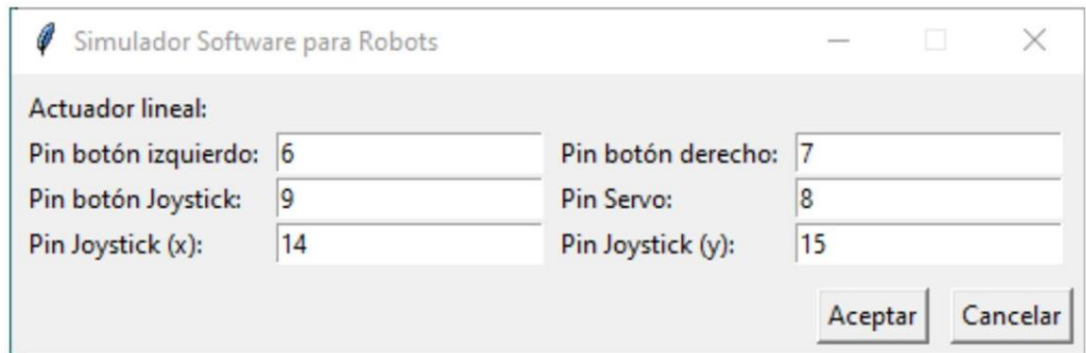


Figura 13 Ventana de configuración – Actuador Lineal

Como podemos ver, si el pin es analógico, como en el caso de las ordenadas del joystick, se muestra el número de pin que correspondería en número. Esto no impide que se pueda escribir el pin con la usual forma A0-A5, de hecho, es recomendable que se haga así para evitar confusiones.

En, tutorials/Tutorial_tablas_DH.pdf se detalla la ventana de configuración del simulador de brazos 3D.

En la cuarta opción vemos el menú ejecutar, que se puede ver en siguiente imagen. Da seis opciones:

- Ejecutar, que comienza la simulación.
- Detener, que la detiene.
- Pausar / Reanudar, que detiene temporalmente la ejecución del sketch sin perder el estado de la simulación.
- Paso a paso, que solo está disponible cuando el simulador se encuentra en estado pausado, y avanza exactamente una sentencia.
- Ampliar, que amplía la imagen de la simulación.
- Reducir, que la reduce.

Ejecutar	Ayuda
Ejecutar	F5
Detener	Ctrl+F5
Pausar / Reanudar	F8
Paso a paso	F10
Ampliar	Ctrl++
Reducir	Ctrl+-

Figura 14 Menú ejecutar

Con respecto a la quinta y última opción, es el menú de ayuda, que incorpora el “Acerca de” de la aplicación y este manual de usuario.

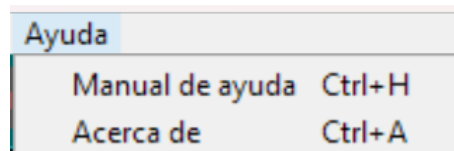


Figura 15 Menú Ayuda

Barra de herramientas

La barra de herramientas de la aplicación se encuentra en la parte superior de la aplicación, justo debajo del menú de esta. Las siguientes imágenes muestran las variaciones de dicha barra.



Figura 16 Barra de herramientas – Actuador Lineal



Figura 17 Barra de herramientas – Robot Móvil



Figura 18 Barra de herramientas – Placa Arduino



Figura 19 Barra de herramientas – Brazo Robótico

En la primera figura, se ve la barra de herramientas del actuador lineal, en la segunda de ambos robots móviles, en la tercera de la placa Arduino y en la cuarta el Brazo Robótico. La única diferencia entre la primera y cuarta con las dos intermedias es que, en el primer caso, solo hay un desplegable, mientras que en el segundo hay dos.

Para el módulo del Robot móvil el segundo desplegable permite seleccionar el circuito que se va a crear.

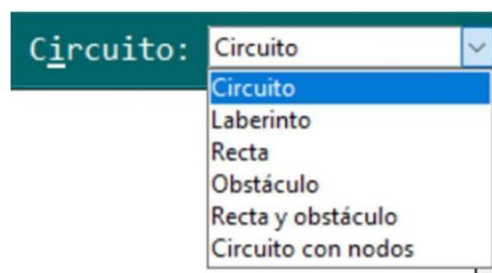


Figura 20 Desplegable selector circuito

Para la placa Arduino, se selecciona el desafío a resolver.



Figura 21 Desplegable selector desafío

Como se puede ver en la segunda imagen, si pasamos por encima de los botones de la izquierda, se mostrará a la derecha de estos la opción que se va a ejecutar. Las opciones de este menú son las siguientes:

- **Ejecutar:** Este botón tendrá dos funcionalidades diferentes.
- Si el usuario se encuentra en el módulo del robot móvil o del actuador lineal, se ejecuta el sketch escrito en la parte del editor de código.
- Si el usuario está en el módulo de la placa Arduino, se comprobará el resultado de la solución al desafío. En este caso, se abrirá una ventana emergente con el feedback proporcionado al usuario, informándole de sus errores, o de su nota en caso de acierto.

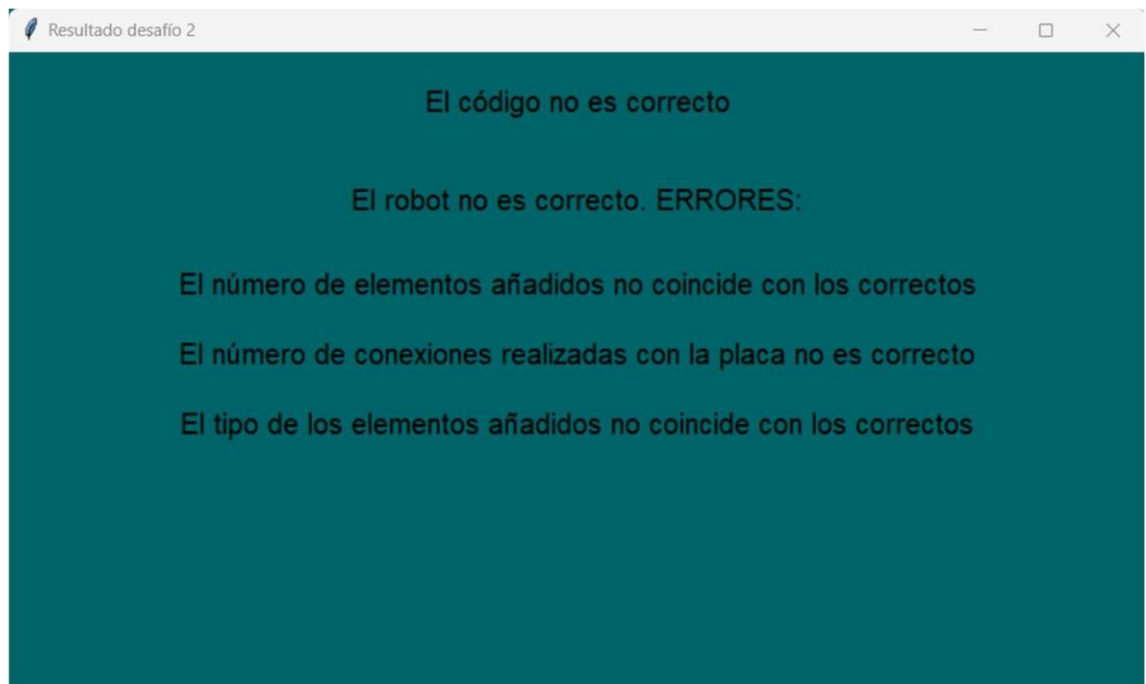


Figura 22 Ventana feedback con errores

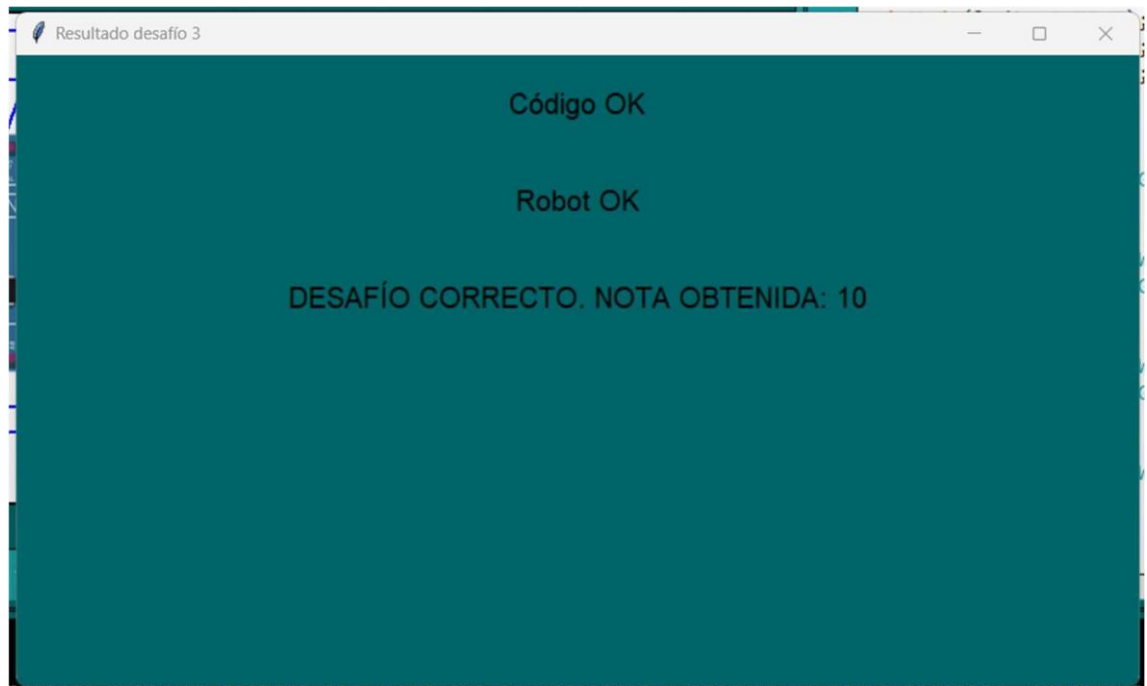


Figura 23 Ventana feedback correcto

- **Detener:** Detiene la ejecución del sketch si este se está ejecutando.
- **Deshacer:** Como en editar, deshace las acciones que se han realizado a la hora de editar el código.
- **Rehacer:** Rehace las acciones que se han realizado a la hora de editar el código.
- **Guardar:** Guarda el sketch de la misma forma que la opción del menú archivo.
- **Importar:** Importa el sketch de la misma forma que la opción del menú archivo.
- **Robot:** Permite cambiar el tipo de robot (y lo que cambie de la interfaz gráfica con él).
- **Circuito (solo robots móviles):** Permite cambiar el circuito que recorrerá el robot móvil.
- **Pausar / Reanudar:** detiene temporalmente el sketch en ejecución sin abortarlo. Al pulsarlo de nuevo se reanuda la ejecución desde el punto exacto de pausa.
- **Paso adelante:** con el sketch pausado, ejecuta una única sentencia del programa y vuelve a pausar. El sistema soporta además breakpoints por línea, de modo que la ejecución se pausa automáticamente al alcanzar una línea marcada.
- **Reiniciar:** combina las acciones de detener y volver a ejecutar el sketch, devolviendo todos los robots a su estado inicial sin necesidad de pulsar manualmente “Detener” y “Ejecutar” en sucesión.

Inmediatamente a la derecha de estos botones se sitúa el indicador de estado de ejecución, una etiqueta de color que refleja en tiempo real la situación del runtime y que está sincronizada con los mensajes de transición que se muestran en la consola del simulador. Los tres estados posibles son “■ DETENIDO” (gris), “► EJECUTANDO” (verde) y “II PAUSADO” (ámbar).

Representación gráfica de los robots

Debajo de la barra de herramientas de la aplicación, hacia la izquierda se encuentra la parte de la representación gráfica de los robots. Su apariencia es la que muestra las siguientes imágenes.

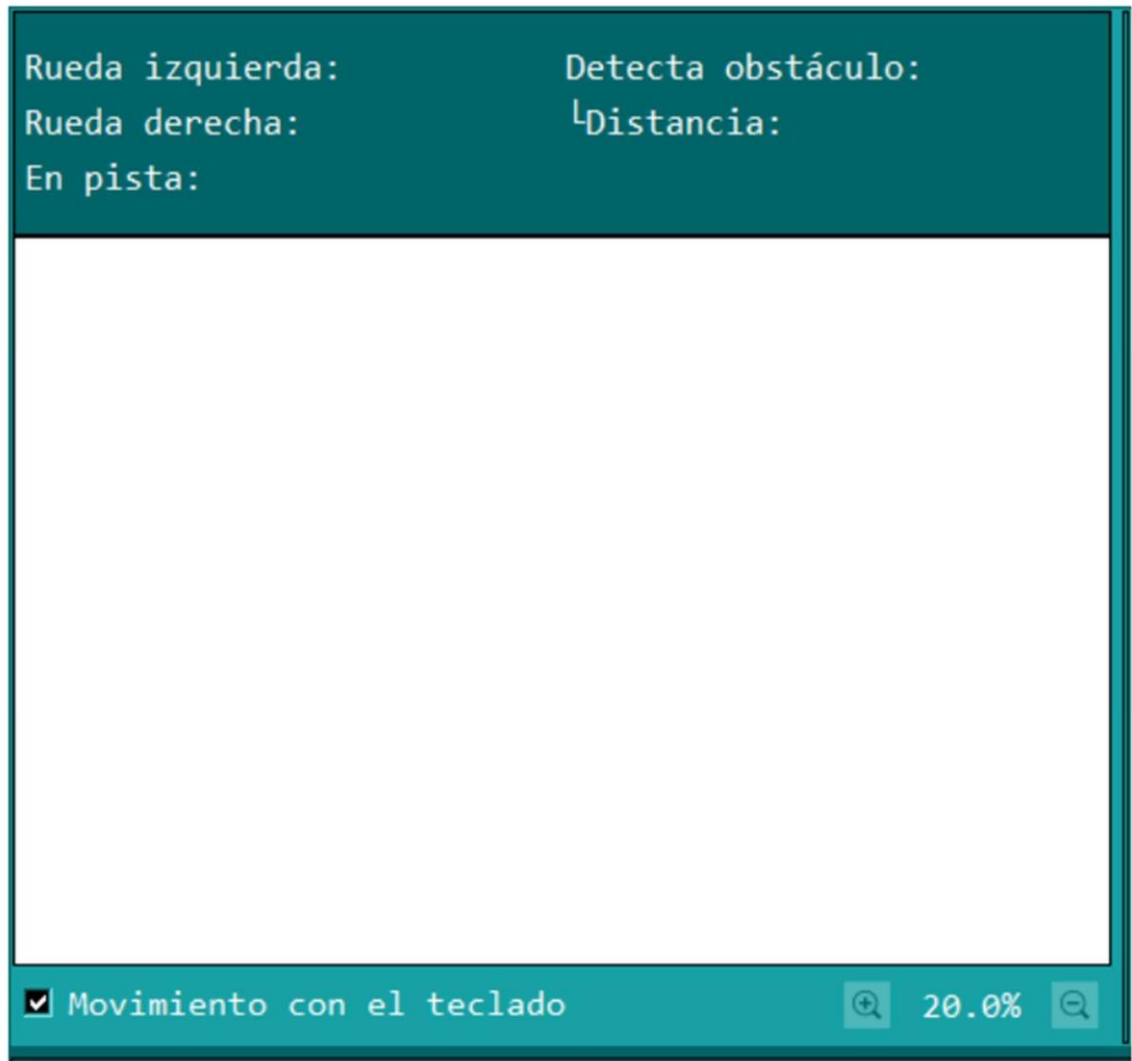


Figura 24 Representación gráfica – Robot móvil – Sin ejecutar

Como vemos en la imagen anterior, la representación gráfica cuenta con un HUD que muestra durante la ejecución los datos de lo que realiza cada componente del robot. Debajo está un recuadro en blanco, que será donde se muestre la simulación del robot. Por último, y abajo del todo, se ve una checkbox para activar o desactivar el movimiento por teclado. Esto es **IMPORTANTE**. Para ejecutar el código (automáticamente), se debe desactivar esta casilla. Si no, el robot estará quieto (a menos que pulsemos las teclas de movimiento WASD, esto es útil para posicionar el robot a nuestro gusto).

La parte derecha de esta barra inferior es la ampliación del dibujo, con el botón izquierdo se acerca y con el derecho se aleja. Se puede realizar lo mismo con el scroll, hacia delante amplía, hacia atrás aleja. La ampliación recomendada en una pantalla de 1920x1080 es 20% con el programa en pantalla completa, de esta manera se podrá ver todo lo que haga el robot.

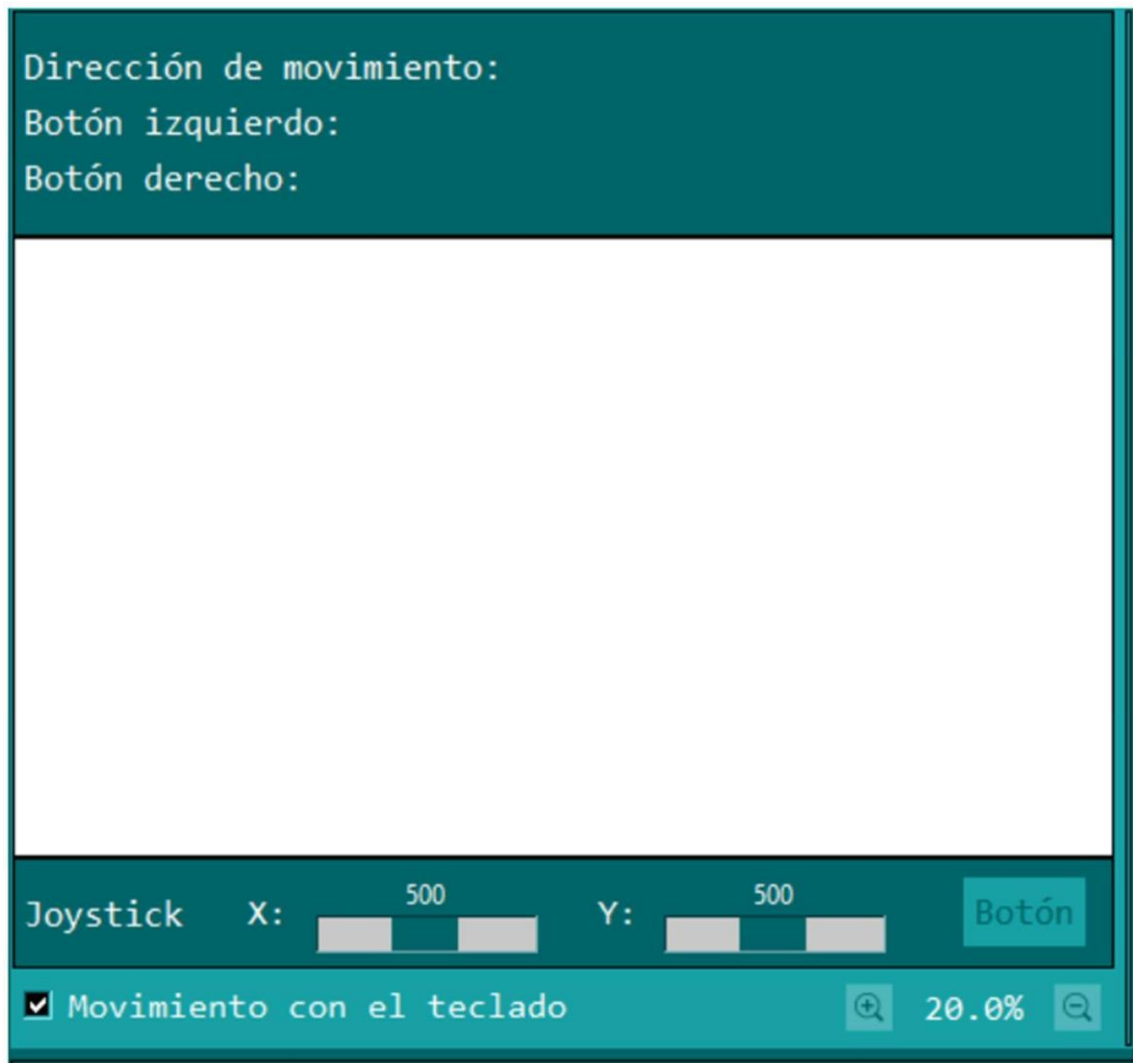


Figura 25 Representación gráfica – Actuador Lineal – Sin ejecutar

Como vemos en la imagen anterior, cambia ligeramente el HUD y se añade una nueva barra. El resto de funcionalidad es la misma que en el caso anterior.

La nueva barra que se ha añadido es la que permite simular el joystick. El botón del joystick funciona pulsando el botón. La dirección del joystick funciona según la siguiente tabla.

0,0	512,0	1023,0
0,512	512,512	1023,512
0,1023	512,1023	1023,1023

Tabla 1 Movimiento del joystick

A continuación, se muestran tres figuras mostrando cada robot en modo ejecución (es decir, representados gráficamente).

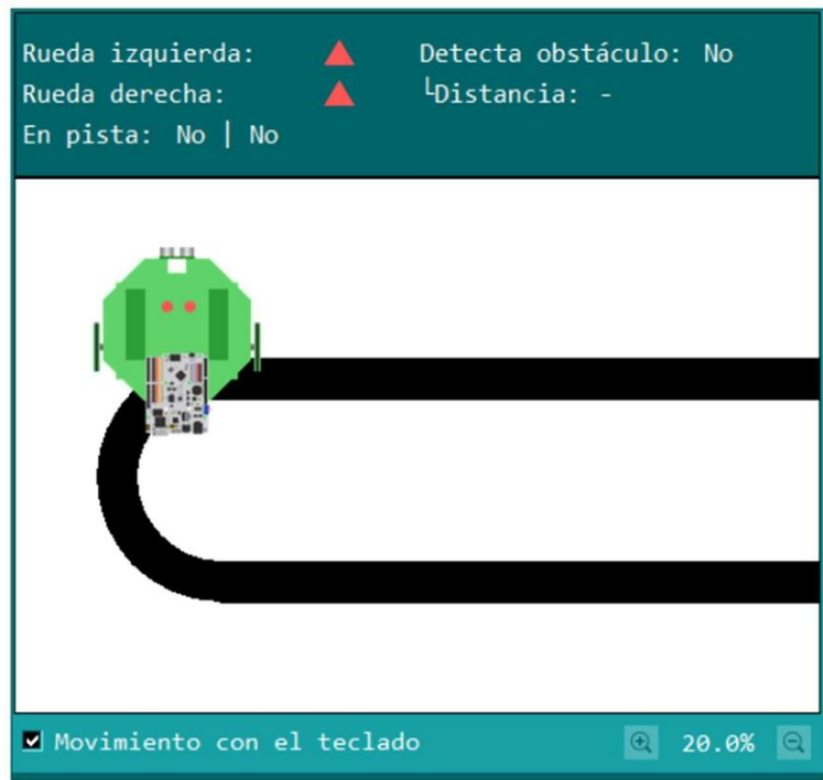


Figura 26 Representación gráfica – Robot móvil 2 – Ejecutando

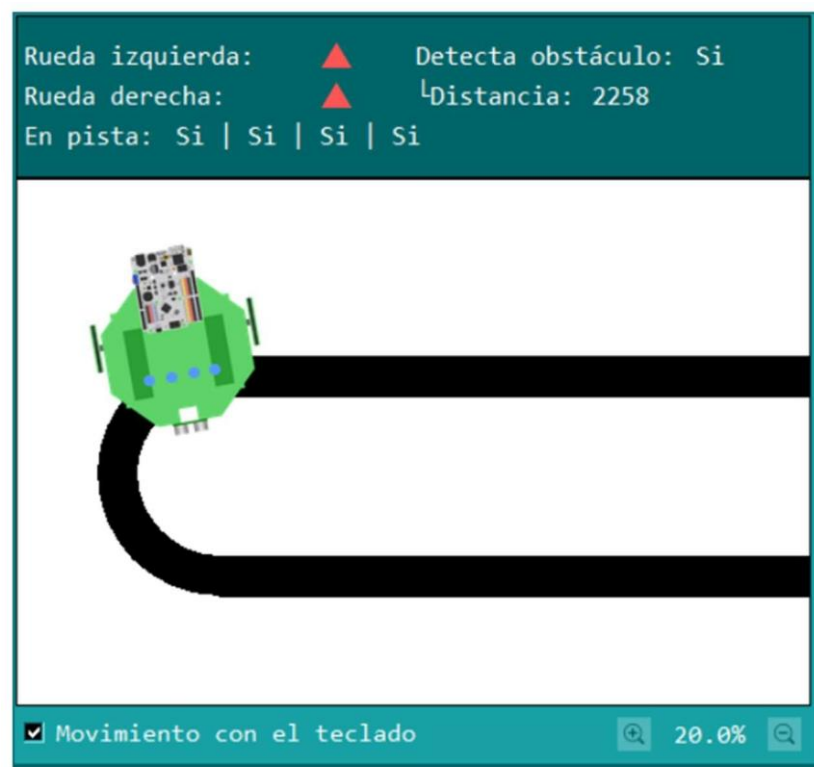


Figura 27 Representación gráfica – Robot móvil 4 – Ejecutando

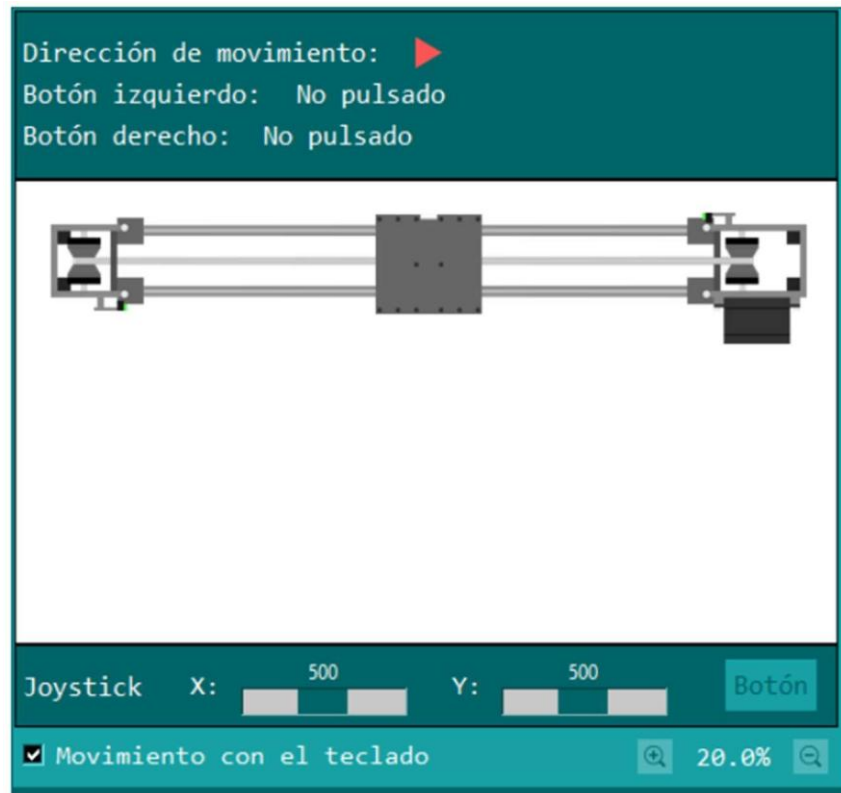


Figura 28 Representación gráfica – Actuador lineal – Ejecutando

En la primera imagen podemos ver cómo se vería el robot móvil de 2 sensores de luz en su representación gráfica. La posición de los sensores de luz se puede ver mediante los círculos rojos. Dichos círculos se colorean de azul si el sensor detecta la pista. Si eso sucede, el HUD mostrará (en el lado correspondiente) si el sensor está en pista o no.

Respecto al HUD, también mostrará en qué dirección se mueve en cada rueda. El color de la flecha mostrará la velocidad del servo (azul – rápido, amarillo – velocidad media, rojo – lento o parado).

El HUD también mostrará si el sensor de ultrasonidos detecta un obstáculo, y si lo detecta mostrará la distancia hacia el obstáculo (no equivalente a la real).

En la segunda imagen se ve que lo único que cambia en cuanto a la representación gráfica del robot móvil de 4 sensores es el número de círculos (sensores) y, en el HUD, se muestra en la parte de en pista lo que detectan los 4 sensores.

Con respecto a la última imagen, en el robot vemos los dos botones que hacen de tope a cada extremo del actuador lineal. Si el bloque del actuador toca un botón, la parte que colisiona con el bloque se vuelve roja, como se muestra en la siguiente imagen, si no, estará verde.

En el caso del HUD, se muestra en qué dirección se mueve el servo y su velocidad de la misma forma que en los robots móviles y también se muestra si los botones están siendo pulsados o no.

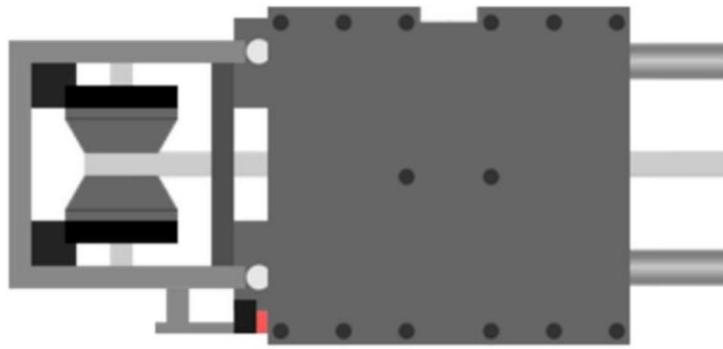


Figura 29 Actuador lineal – El botón tope está pulsado

Por otro lado, la representación gráfica del módulo de la placa Arduino es diferente a los anteriores.

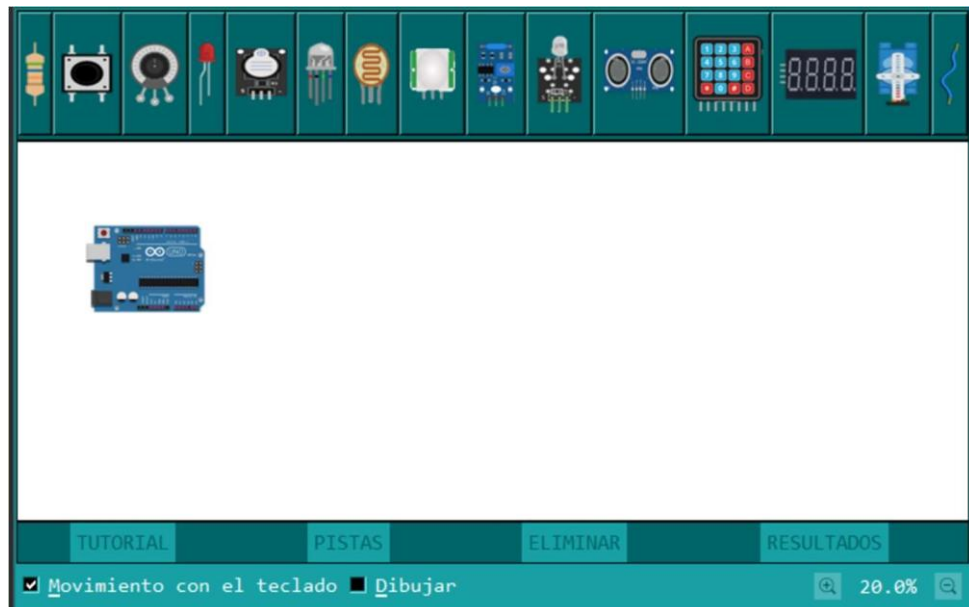


Figura 30 Representación gráfica – Placa Arduino

Como podemos ver en la imagen anterior, a simple vista se aprecian las principales diferencias. Una de ellas es que en el apartado del HUD, se han colocado los botones para los elementos. También podemos ver cómo, sin estar ejecutándose, tenemos elementos dentro del cuadro de dibujo. Inicialmente solo aparecerá la placa, igual que vemos en la imagen, pero posteriormente el usuario puede añadir nuevos componentes.

Otra diferencia es la barra de botones que encontramos en la parte inferior del cuadro. Estos botones ofrecen diferentes funcionalidades al usuario.

- Tutorial: este botón abre un archivo en formato pdf con toda la información teórica relativa a Arduino y a los circuitos y componentes que puede necesitar el usuario para completar los desafíos. También recoge ejemplos de conexiones y código.



Figura 31 Tutorial

- Pistas: al pulsar sobre este botón, el usuario verá una serie de pistas relativas al desafío que está resolviendo. Cada vez que se consulten las pistas se mostrará una pista más que la anterior, hasta que se hayan mostrado todas.

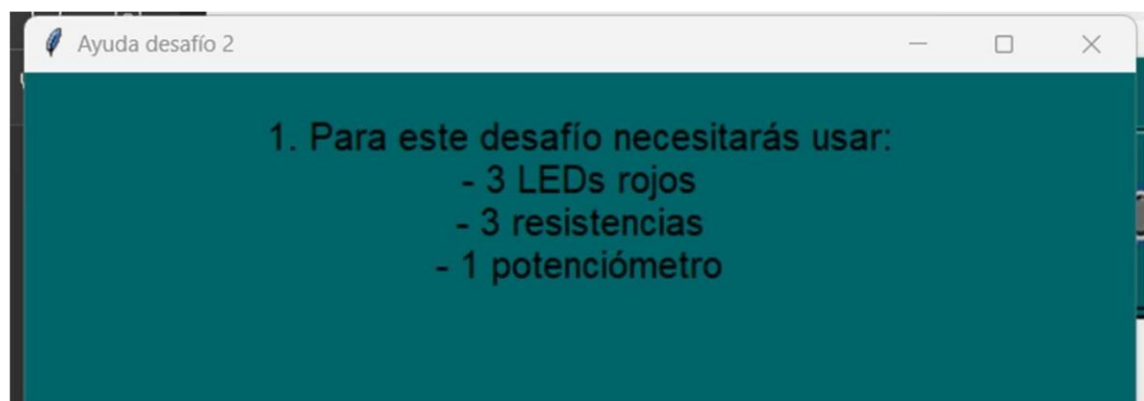


Figura 32 Primera pista de un desafío

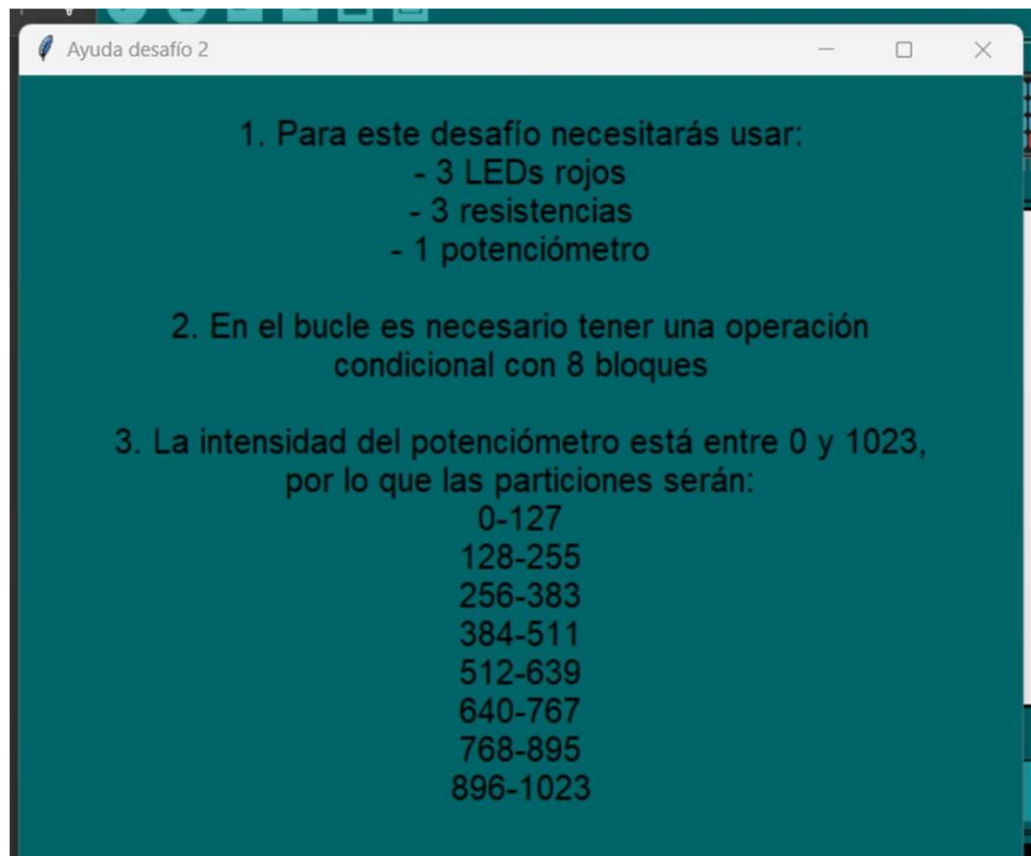


Figura 33 Todas las pistas de un desafío

- Eliminar: al pulsar este botón se reinicia el cuadro de dibujo.
- Resultados: cuando se selecciona este botón se abre una ventana emergente que muestra al usuario el histórico de sus resultados.

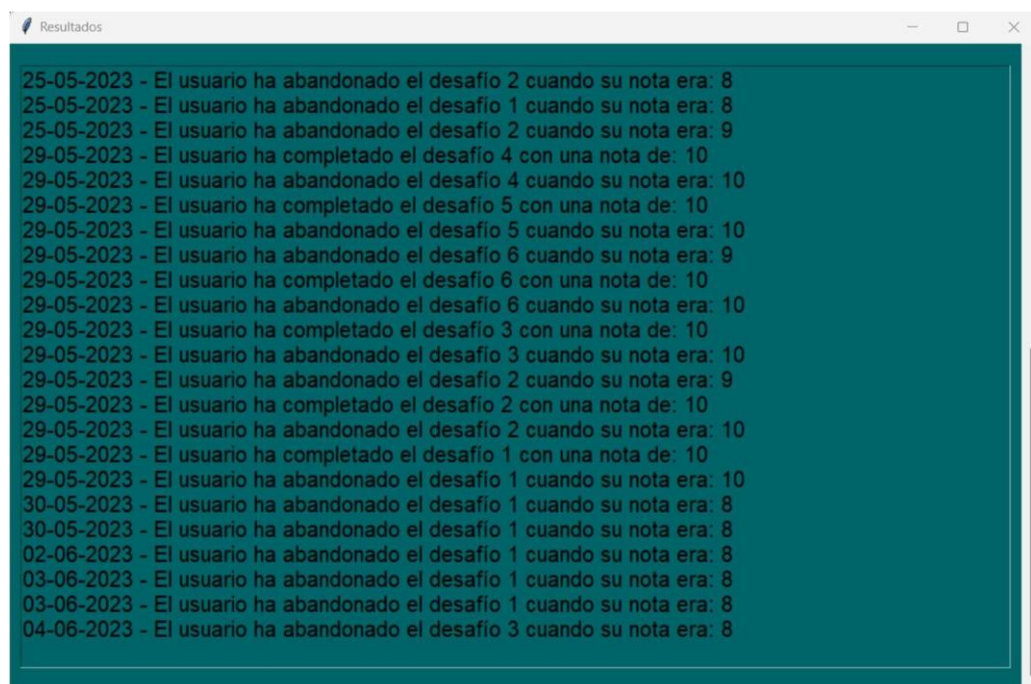


Figura 34 Histórico de resultados de un usuario

La barra que encontramos debajo de estos botones es igual que para los modelos anteriores, con la excepción del check de “Dibujar”. Este debe ser marcado por el usuario para comenzar a colocar y unir elementos.

Al seleccionar el robot **TinkerKit Braccio**, el área de trabajo cambiará automáticamente de la vista 2D estándar a un entorno tridimensional. Este modo 3D incorpora herramientas, controles e interfaces específicos para operar el brazo robótico:

Panel de Información

El HUD estándar se oculta y en su lugar se despliega el panel de información para brazos 3D. Esta interfaz de telemetría le permite monitorizar en tiempo real los siguientes parámetros:

- **Grados de libertad:** Número total de articulaciones activas.
- **Estado articular (J1-J6):** Ángulos actuales de cada articulación y sus límites de movimiento físico.
- **Posición espacial:** Coordenadas cartesianas (**XYZ**) del efector final.
- **Semáforo de seguridad:** Indicador visual para alertar sobre estados anómalos o de riesgo.

Para explorar el entorno 3D, utilice los controles del ratón de la siguiente manera:

- **Orbitar:** Mantenga pulsado el **botón izquierdo** y arrastre el ratón para rotar la cámara alrededor del brazo.
- **Desplazar (Pan):** Mantenga pulsado el **botón derecho** y arrastre para mover el punto de mira a lo largo del plano.
- **Zoom:** Gire la **rueda del ratón** para acercar o alejar la vista.

Adicionalmente, puede utilizar los botones de *Vista rápida* situados sobre el área de dibujo para saltar instantáneamente entre tres perspectivas predefinidas: **Libre**, **Caballera** e **Isométrica**. O también para seleccionar controles alternativos de órbita, desplazamiento y zoom.

Puede personalizar el aspecto del robot accediendo a la ventana de configuración del brazo. El área de trabajo admite tres modos visuales intercambiables:

- **braccio_exact:** Renderiza las mallas 3D (STL) del modelo del brazo (Precargadas en el simulador). Es el modo de mayor fidelidad visual.
- **auto_generic:** Genera una representación geométrica de forma procedural, basada estrictamente en los parámetros cinemáticos de Denavit-Hartenberg (DH).
- **skeleton:** Oculta la geometría externa y muestra únicamente el esqueleto cinemático. Resulta especialmente útil para tareas de análisis y depuración.

Para conocer en profundidad todas las herramientas del entorno 3D —incluyendo la navegación avanzada, los controles deslizantes de cinemática directa, la cinemática inversa y las opciones de visualización (trayectoria, rangos, ejes)— consulta el apartado **¡Error! No se encuentra el origen de la referencia. ¡Error! No se encuentra el origen de la referencia..**

Editor de código

A la derecha de la representación gráfica está el editor de código. La siguiente imagen muestra cómo se estructura.

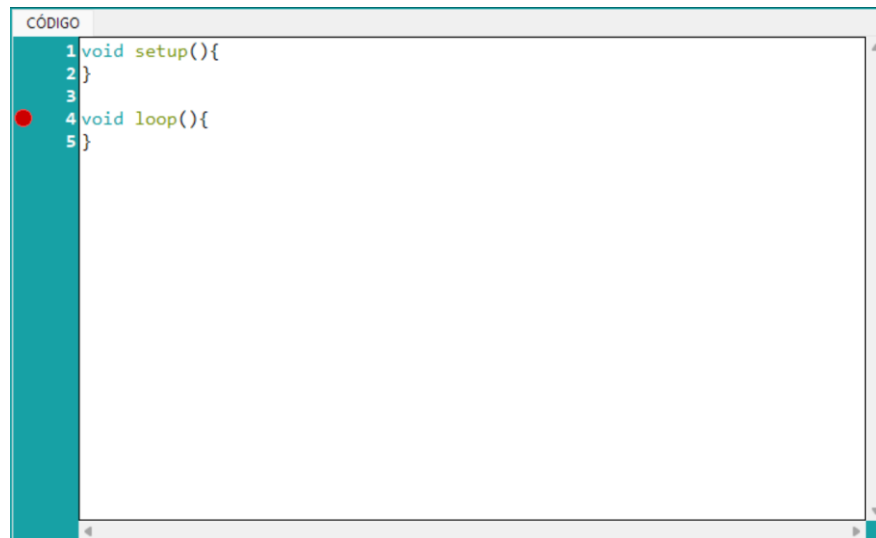


Figura 35 Editor de código

El editor tiene coloreado sintáctico (el mismo que Arduino IDE). Además, a la izquierda se muestra el número de línea correspondiente (esto facilita la búsqueda de errores y advertencias).

Se puede navegar a través del código usando las barras de scroll lateral y vertical. Se volverán activas en caso de ser necesarias y se pueden utilizar usando la rueda del ratón. El editor no autocompleta el código.

Adicionalmente el espacio a la izquierda del conteo de líneas permite marcar las líneas de break para depuración línea a línea.

Consola de entrada/salida

Debajo de los dos últimos componentes se sitúa la consola, que se muestra en la siguiente imagen.



Figura 36 Consola

En la parte superior izquierda tenemos la consola como tal. Mostrará la salida del sketch, con un código de colores en el cual rojo es un error, amarillo una advertencia y blanco un mensaje normal. Estos mensajes se pueden filtrar con los checkboxes de la derecha (se pueden usar a la vez varios).

En la parte inferior hay un campo de texto que es el que se debe usar si se quiere hacer una entrada por consola. La entrada se envía usando el botón enviar.

Cuando el módulo seleccionado es el de la placa Arduino, en la consola se mostrará el enunciado del desafío que debe resolver el usuario. Aparecerá como se muestra en la siguiente imagen.

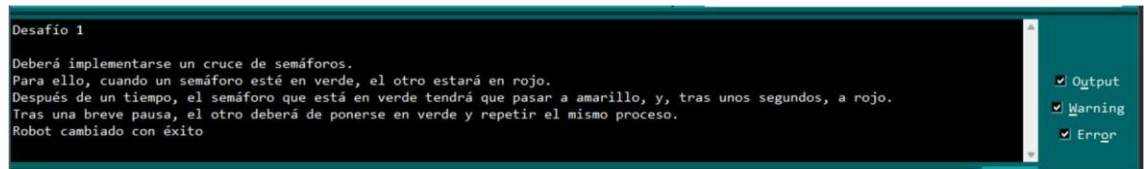


Figura 37 Consola con desafío

Creación de circuito

Desde el módulo de la placa Arduino, el sistema propone al usuario un desafío a resolver, como podemos ver en la consola inferior. En este el momento el usuario debe realizar dos procesos.

- Crear el circuito correcto de cara a lo pedido en el desafío.
- Introducir el código del programa para que funcione como se espera.

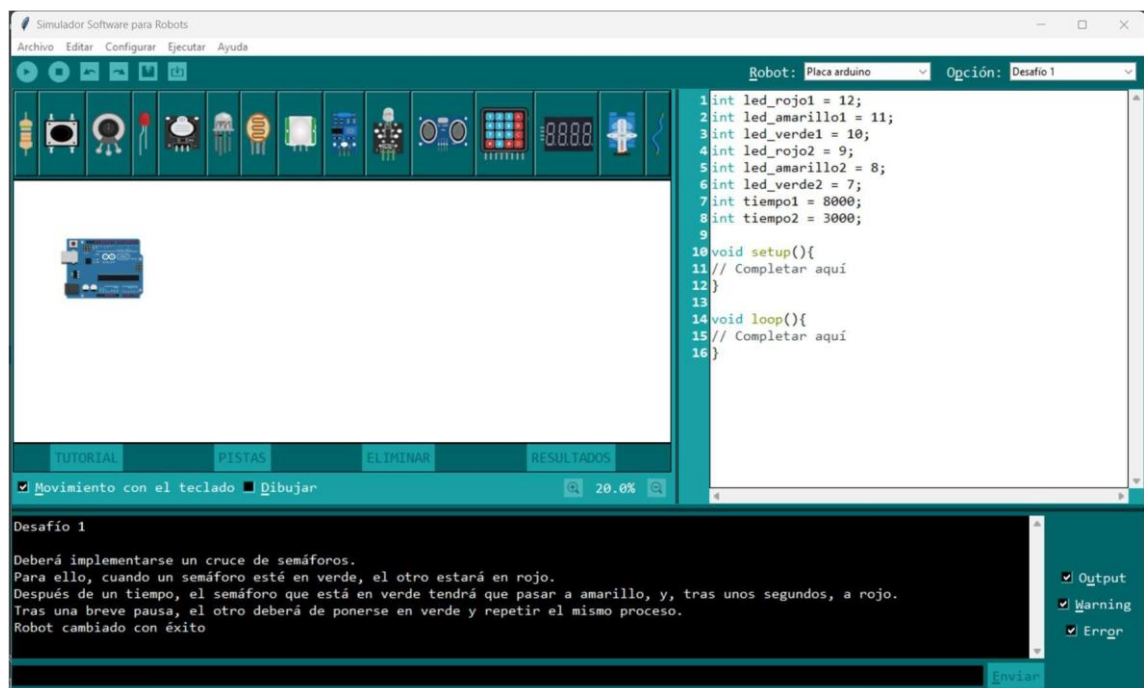


Figura 38 Visión interfaz placa Arduino

Para crear el circuito, lo primero que debe hacer el usuario es seleccionar la opción de dibujar, de esta forma podrá comenzar a colocar elementos en el cuadro de dibujo.

Para colocar elementos, el usuario debe seleccionar el componente a dibujar de la barra encima del cuadro de dibujo. Después de esto, pulsando en cualquier lugar sobre el cuadro el componente se dibujará en ese punto.



Figura 39 Elemento colocado en el cuadro de dibujo

Para conectar dos componentes, existen dos opciones.

- Si se pulsa sobre uno de los botones de uno de los pines de un componente, cuando se pulse sobre otro distinto, se conectarán ambos pines con una línea recta.

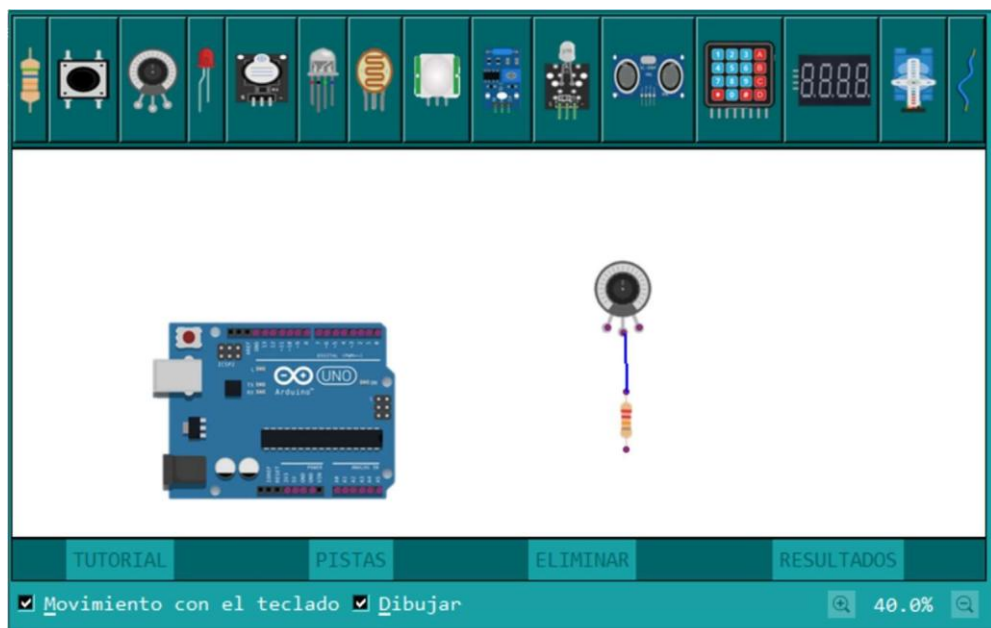


Figura 40 Conexión de componentes (I)

- Por otro lado, si se quiere crear un circuito con una forma determinada, se debe seleccionar la opción del cable de la botonera superior (primer botón por la derecha). Tras esto, seleccionando como punto de partida uno de los botones de los pines de un componente, cada vez que se haga clic sobre un punto del cuadro de dibujo, se creará una línea recta entre estos dos puntos. Para cerrar el circuito, se deberá pulsar otro botón de pin.

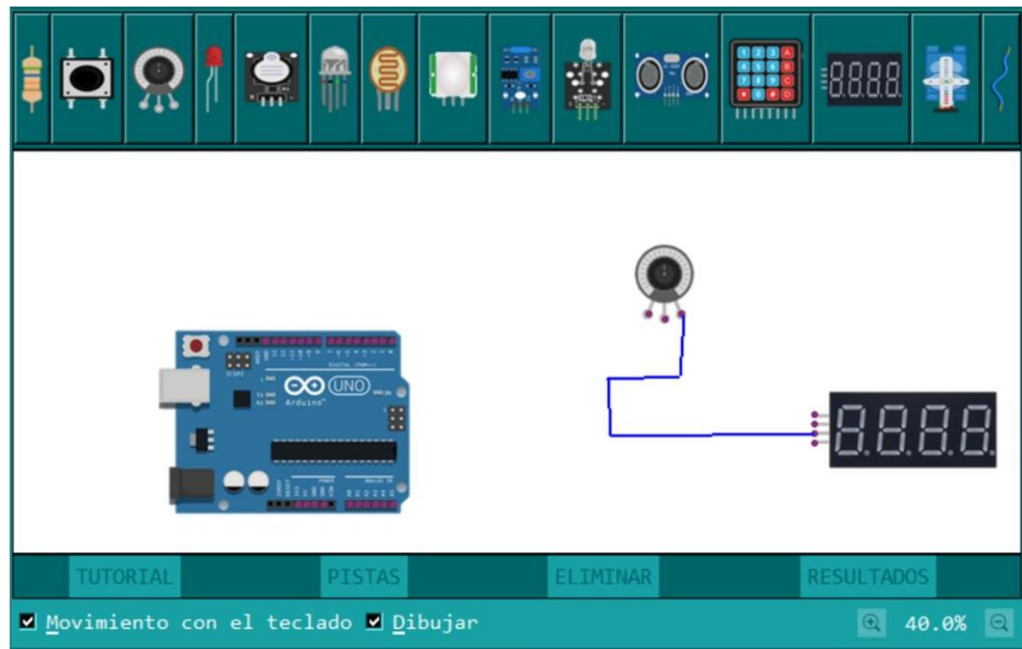


Figura 41 Conexión de componentes (II)

Otras características para tener en cuenta

Ejecución de Programas y Respuesta de la Interfaz

La interfaz gráfica está diseñada para permanecer fluida y completamente interactiva durante toda la ejecución de su sketch. No es necesario esperar a que finalicen los bucles o las pausas programadas para seguir operando el entorno.

- **Interacción en tiempo real:** Durante la ejecución, usted puede utilizar libremente los controles de cámara, los deslizadores (sliders) y la barra de herramientas.
- **Detención inmediata:** Al pulsar el botón Parar (Stop), el programa se interrumpirá de forma segura en menos de medio segundo, independientemente de en qué punto del código se encuentre.

Solución de Problemas: Configuración de Pines

Una gran parte de los errores durante la ejecución provienen de una mala asignación en las conexiones lógicas del robot.

- **Buenas prácticas:** Se recomienda revisar siempre la configuración de los pines antes de pulsar el botón de ejecutar para evitar fallos de ejecución.
- **Advertencia sobre la modificación de pines:** Los pines por defecto del programa se almacenan en el archivo robot_data.json. Si bien es posible abrir este archivo para cambiarlos manualmente, no es recomendable editarlo a menos que tenga conocimientos avanzados sobre la estructura de datos del robot.

Simulador 3D

Este apartado describe el uso del módulo de simulación 3D del brazo TinkerKit Braccio, cubriendo la vista 3D, los controles de cámara, el posicionamiento manual de articulaciones, la cinemática inversa, la persistencia de configuraciones y la escritura de scripts Arduino que controlen el brazo.

Vista 3D del brazo robótico

Al seleccionar el robot Braccio, el área de dibujo central muestra la escena tridimensional del brazo. El modelo renderiza en tiempo real las seis piezas ensambladas del TinkerKit Braccio (base, hombro, codo, muñeca vertical, muñeca rotatoria y pinza) según los ángulos articulares actuales. La escena incluye una rejilla de referencia en el plano del suelo y los ejes del sistema de coordenadas del robot.

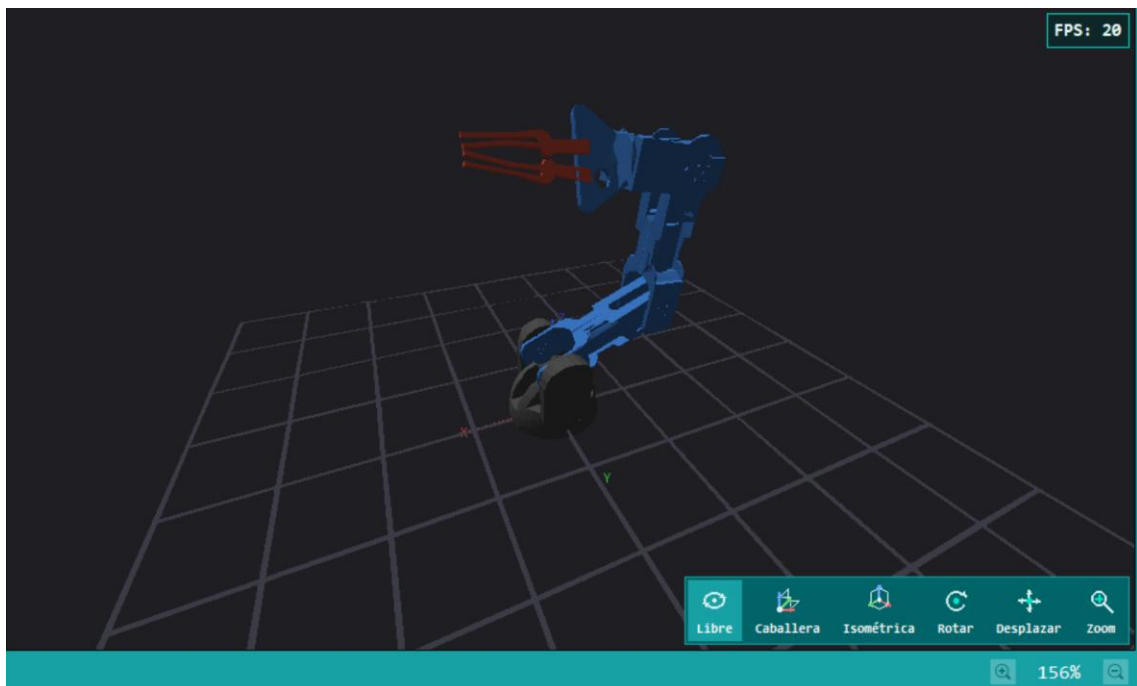


Figura 42 Vista 3D del TinkerKit Braccio

Panel de información del brazo

El panel izquierdo “Brazo Robótico 3D” muestra en tiempo real tres bloques de información:

- **Efector final:** coordenadas X, Y, Z en milímetros del punto de contacto del efector respecto al sistema de referencia de la base del robot.
- **Articulaciones:** ángulo actual de cada articulación J1-J6 en grados y los límites hardware entre corchetes. El ángulo se actualiza con cada movimiento, ya sea manual, por script o por cinemática inversa.
- **Estado:** indicador de seguridad que muestra “OK” en verde o “Singularidad” en naranja cuando el algoritmo de cinemática inversa alcanza una configuración degenerada del brazo.

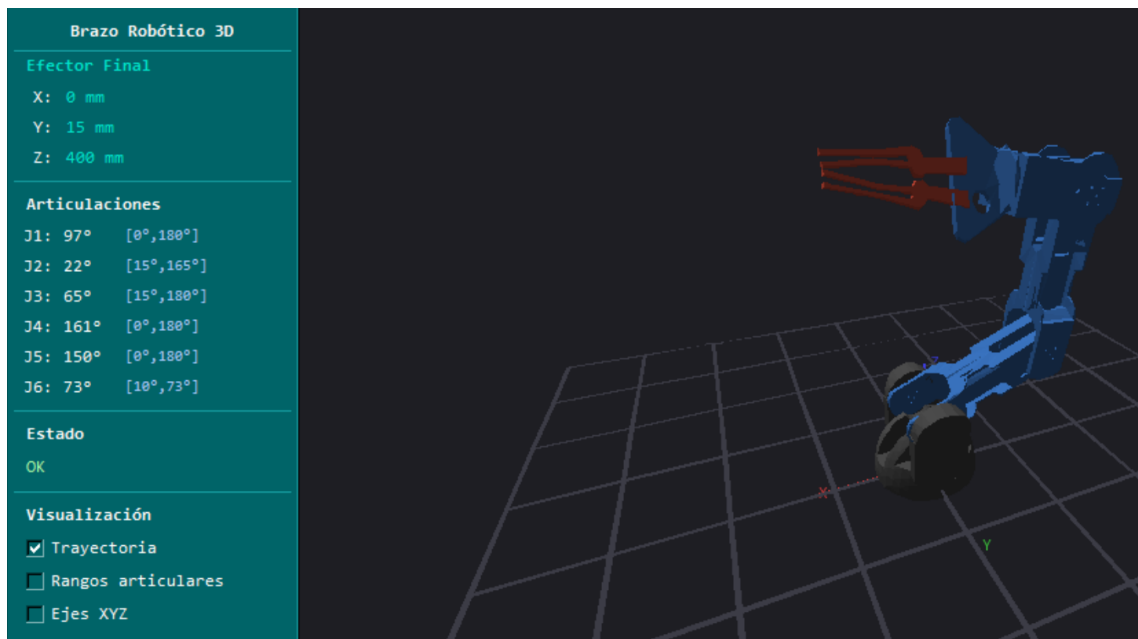


Figura 43 Panel de información con coordenadas del efector, ángulos articulares y estado de seguridad

En la parte inferior del panel se encuentran los toggles de visualización, descritos en el apartado 0 Opciones de visualización.

Posicionamiento manual de articulaciones (cinemática directa)

La pestaña “CONTROL MANUAL” del panel derecho expone el control de cinemática directa mediante seis sliders horizontales etiquetados J1 a J6, uno por articulación del Braccio. Cada slider trabaja en grados dentro de los límites mecánicos reales del servo correspondiente, mostrando junto al título el valor numérico actual.

Las articulaciones se identifican en el siguiente orden: J1 = base (giro horizontal del conjunto), J2 = hombro (inclinación hacia delante y atrás), J3 = codo, J4 = muñeca vertical, J5 = muñeca rotatoria, J6 = pinza (apertura y cierre).

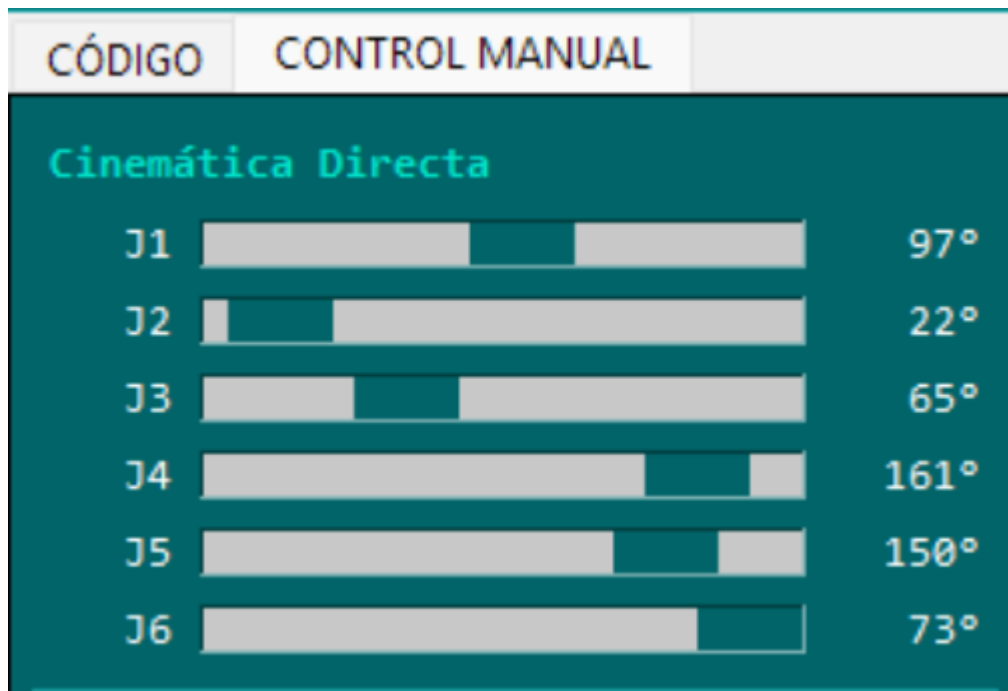


Figura 44 Pestaña “CONTROL MANUAL” con los sliders de cinemática directa J1-J6

Cinemática inversa

La sección “Cinemática Inversa” de la pestaña “CONTROL MANUAL” permite especificar una posición objetivo del efector final en coordenadas cartesianas (X, Y, Z en milímetros) y pulsar el botón “Confirmar” para que el simulador calcule la configuración articular que la satisface.

El algoritmo empleado es un solver multi-arranque que combina Jacobiano analítico, búsqueda en línea (line search) por iteración y unidades internas mixtas (radianes para articulaciones rotacionales, milímetros para prismáticas):

- Cuando el objetivo no es estrictamente alcanzable, el solver aplica la mejor aproximación encontrada en lugar de cancelar el movimiento. La fachada Motor3DApi.solve_ik usa 150 iteraciones máximas por semilla y reporta la convergencia a partir de 5 mm, aunque el solver sigue refinando internamente hasta cerca de 1 mm.
- Si la posición objetivo es alcanzable, los sliders J1-J6 se desplazan animadamente hasta la solución; en caso contrario, se muestra un mensaje informativo de la imposibilidad de alcanzar las coordenadas pedidas y los valores articulares se alteran posicionándose en una configuración posible lo más cercana a la pedida.

El botón “Reset Cámara” de la pestaña restaura la posición orbital inicial, recalculando automáticamente la distancia orbital óptima para el modelo activo.

Cinemática Inversa

X: 0 Y: 0 Z: 400

Confirmar Posición

Reset Cámara

IK convergida (error=1.0 mm)

Figura 45 Campos de cinemática inversa con coordenadas objetivo y mensaje de resultado

Navegación de cámara

La cámara de la vista 3D es orbital y admite tres tipos de movimiento en el modo Normal (predeterminado): el botón izquierdo del ratón (LMB) pulsado y arrastrado rota la cámara alrededor del brazo; el botón derecho del ratón (RMB) pulsado y arrastrado desplaza (pan) el punto de mira; y la rueda del ratón realiza zoom acercando o alejando la cámara orbitalmente.

Los botones de vista rápida sobre el área de dibujo permiten saltar a perspectivas predefinidas:

- **Libre:** restablece la orientación por defecto de la cámara.
- **Caballera:** preset de cámara en perspectiva con yaw 240°, pitch 18° y distancia 950 mm que reproduce visualmente la sensación oblicua característica de la proyección caballera sin emplear proyección ortográfica real.
- **Isométrica:** proyección ortográfica isométrica con yaw 45° y pitch 32°.

También junto a las vistas predefinidas se incluyen opciones de ajuste de los modos alternativos de movimiento que configura la acción del botón izquierdo del ratón, permitiendo usar un solo botón, el click izquierdo del ratón concretamente.

Además, el sistema presenta la posibilidad de moverse orbitalmente por la escena usando las teclas del teclado WASD.

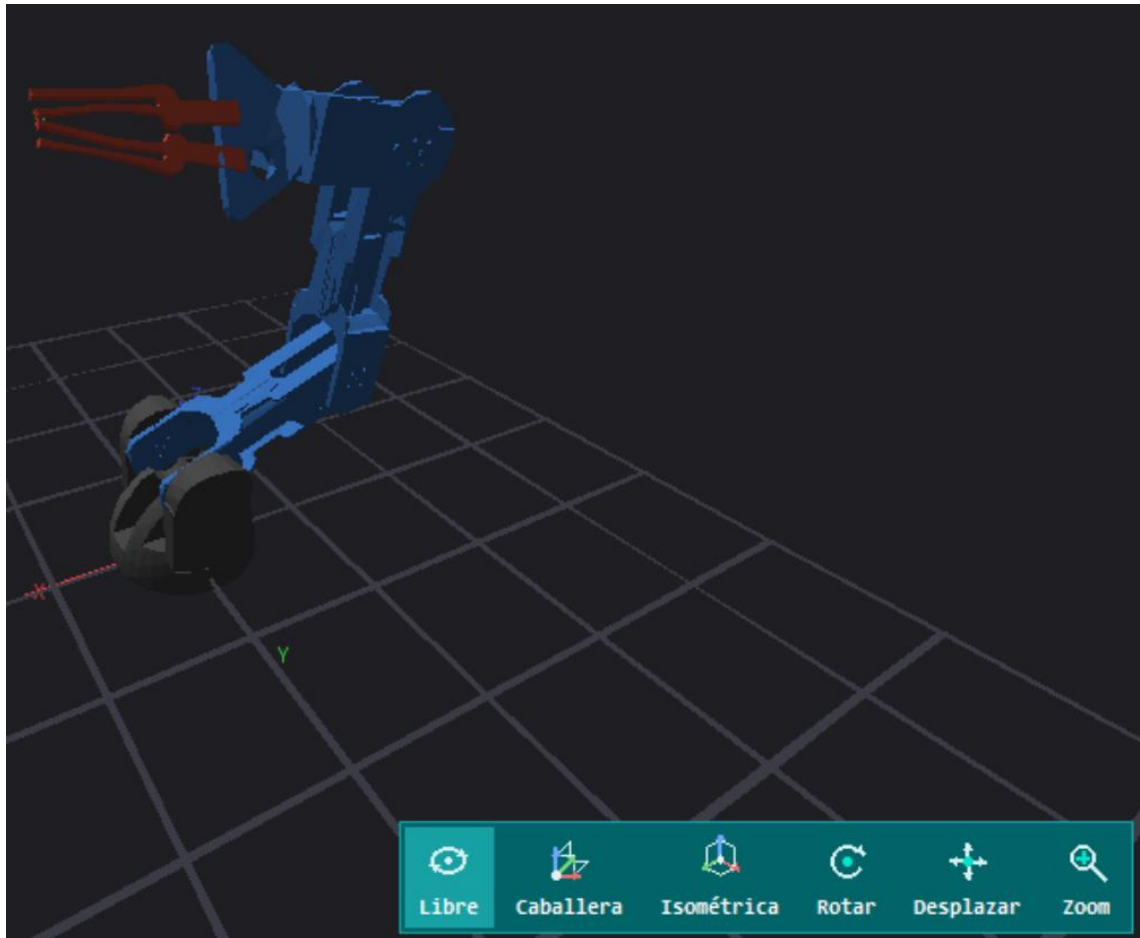


Figura 46 Botones de vista rápida de cámara y modos de movimiento en la pestaña de control

Opciones de visualización

El panel izquierdo contiene en su parte inferior tres casillas de verificación que se activan y desactivan en tiempo real sin interrumpir la simulación:

- **Trayectoria:** activa el trazado de la ruta del efector final. Cada vez que el efector se mueve, se dibuja una línea en el espacio 3D que acumula el recorrido histórico del extremo del brazo. Útil para analizar la curva descrita por scripts de movimiento complejos.
- **Rangos articulares:** superpone sobre el modelo 3D arcos semitransparentes que representan el rango angular posible de cada articulación. Permite identificar visualmente qué articulaciones están cerca de su límite mecánico.
- **Ejes articulares:** dibuja los ejes de rotación de cada articulación como vectores coloreados (rojo = X, verde = Y, azul = Z en el convenio Denavit-Hartenberg). Útil para comprender la orientación de cada eslabón.

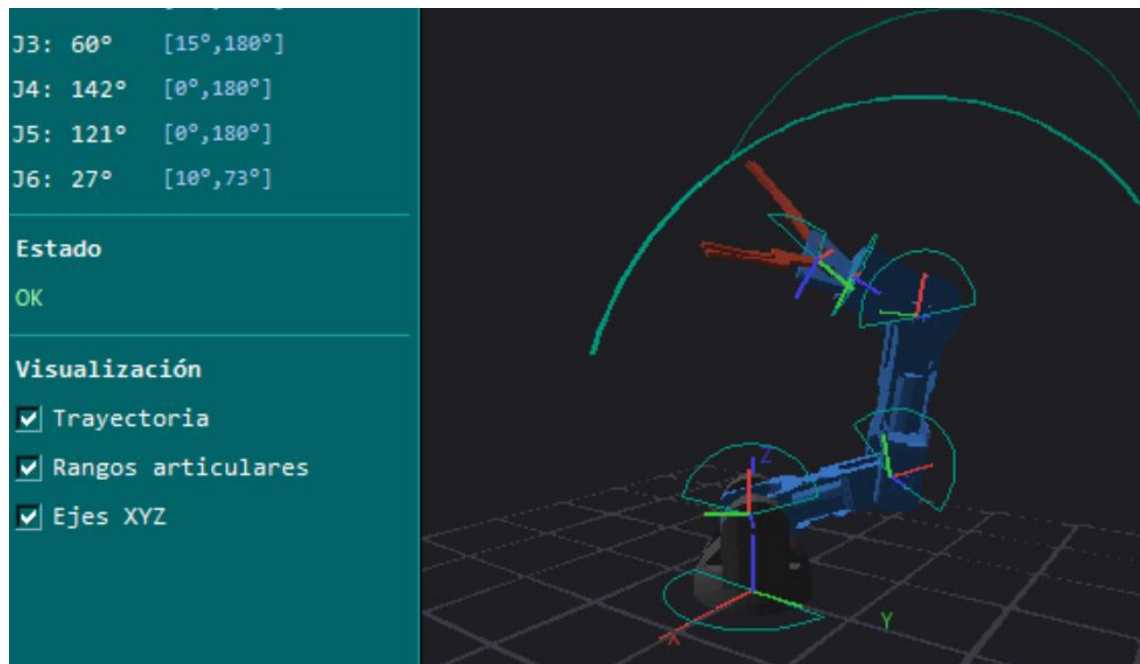


Figura 47 Vista 3D con trayectoria del efector, rangos articulares y ejes de rotación activados

5. Manual de Calibración del Braccio físico

Los servomotores del TinkerKit Braccio no son perfectamente lineales: por las tolerancias de fabricación y los engranajes internos, ordenar 90 al servo no coloca la articulación exactamente a 90 grados mecánicos, y el error varía a lo largo del recorrido y entre unidades. Para que la simulación reproduzca fielmente el brazo del laboratorio, el simulador aplica una tabla de calibración por articulación que relaciona el valor digital enviado al servo (0–180) con el ángulo mecánico real medido. Este manual explica cómo obtener esa tabla con el sketch de medición incluido en el proyecto y cómo aplicarla.

Dónde vive la calibración y cuándo se aplica

La tabla efectiva se almacena en el campo `servo_calibration` del preset `simulator/assets/presets/braccio_tinkerkit.json`: por cada articulación (M1 a M6), una lista de tres puntos `[valor_digital, ángulo_real]`. La clase `Arm3DLayer` (`simulator/graphics/layers.py`) la lee del modelo activo; si el preset no definiera el campo, recurre a la tabla por defecto `DEFAULT_BRACCIO_SERVO_CALIBRATION` declarada en esa misma clase. Entre puntos se interpola linealmente, y fuera del rango se satura al punto extremo.

```

1  {
2    "dof": 6,
3    "link_lengths": [2.0, 125.0, 125.0, 0.0, 0.0, 0.5],
4    "joints": [0.0, 0.0, 0.0, 0.0, 0.0, 0.0],
5    "joint_limits": [
6      [-92.0, 78.0],
7      [-75.0, 75.0],
8      [-50.0, 110.0],
9      [-70.0, 110.0],
10     [0.0, 162.0],
11     [-80.0, -17.0]
12   ],
13   "joint_types": ["R", "R", "R", "R", "R", "R"],
14   "servo_pins": [11, 10, 9, 6, 5, 3],
15   "servo_calibration": [
16     [[0.0, 168.0], [90.0, 83.0], [180.0, -2.0]],
17     [[165.0, 15.0], [75.0, 90.0], [15.0, 150.0]],
18     [[20.0, 200.0], [130.0, 90.0], [180.0, 40.0]],
19     [[0.0, 20.0], [70.0, 90.0], [180.0, 200.0]],
20     [[0.0, 90.0], [100.0, 180.0], [180.0, 252.0]],
21     [[10.0, 10.0], [40.0, 40.0], [73.0, 73.0]]
22   ],
23   "dh_rows": [
24     {"theta": -180.0, "d": 72.0, "a": -2.0, "alpha": 90.0},
25     {"theta": 90.0, "d": 0.0, "a": 125.0, "alpha": 0.0},
26     {"theta": 0.0, "d": 0.0, "a": 125.0, "alpha": 0.0},
27     {"theta": -90.0, "d": 0.0, "a": 0.0, "alpha": 90.0},
28     {"theta": 90.0, "d": -60.0, "a": 0.0, "alpha": 180.0},
29     {"theta": -90.0, "d": 126.55, "a": 0.5, "alpha": 0.0}
30   ],
31   "visual": {
32     "mode": "braccio_exact",
33     "theme": "default",
34     "sizes": {}
35   }
36 }

```

Figura 48 El campo `servo_calibration` del preset `braccio_tinkerkit.json`

La calibración solo se activa con el Braccio predefinido: cuando `Motor3DApi.uses_legacy_servo_degrees()` devuelve `True` (preset `braccio_tinkerkit` con modo visual `braccio_exact`). Los brazos genéricos definidos por el usuario usan la conversión estándar ($\text{ángulo_modelo} = \text{valor_servo} - 90$) sin tabla. Importante: los valores actualmente incluidos proceden de mediciones preliminares; el procedimiento de este manual es el camino para sustituirlos por una calibración sistemática completa.

Material necesario

- Brazo TinkerKit Braccio montado y conectado a su placa Arduino mediante el shield, con su alimentador oficial (los servos exigen más corriente de la que da el USB).
- PC con el IDE de Arduino y cable USB.
- Transportador de ángulos (o goniómetro) y, a ser posible, una varilla o referencia recta que prolongue el eslabón para leer mejor el ángulo. (También se puede hacer a ojo de buen cubero)
- La plantilla de registro siguiente (una fila por articulación; se rellena con los pares valor digital → ángulo real medido en los tres puntos de referencia).

Motor	Articulación	Módulo: digital → real	Neutro: digital → real	Máximo: digital → real
M1	base	→ ____°	→ ____°	→ ____°
M2	hombro	→ ____°	→ ____°	→ ____°
M3	codo	→ ____°	→ ____°	→ ____°
M4	muñeca vertical (pitch)	→ ____°	→ ____°	→ ____°
M5	muñeca rotativa (roll)	→ ____°	→ ____°	→ ____°
M6	pinza	→ ____°	→ ____°	→ ____°

Tabla 2 Plantilla de registro de calibración usada

- La placa Arduino UNO con el shield TinkerKit sobre él y con los seis servos, el alimentador externo y el cable USB.

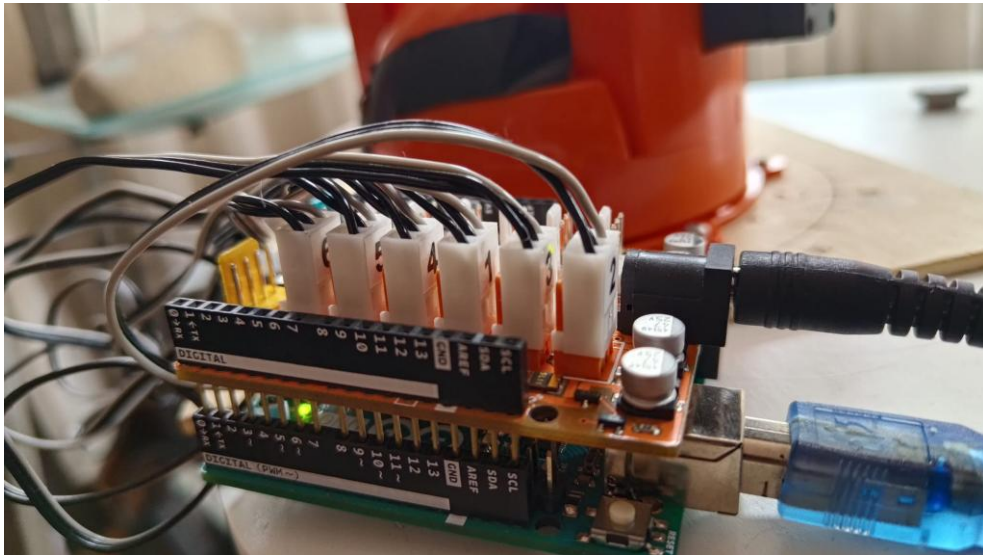


Figura 49 shield TinkerKit sobre el Arduino UNO con los seis servos, el alimentador externo y el cable USB.

El sketch de medición `braccio_medicion_real.ino`

El proyecto incluye en codes/ el sketch `braccio_medicion_real.ino`, diseñado para tomar las medidas sobre el Braccio físico. Implementa un menú interactivo por la consola Serie a 9600 baudios. Sus comandos:

Comando	Efecto
help	Reimprime la ayuda con todos los comandos.
protocol	Imprime los protocolos guiados de medición de límites y de holgura (backlash).
pose	Imprime la pose actual: motor seleccionado, paso, y los seis valores M1–M6 enviados a los servos.
reset	Devuelve el brazo a la pose base segura (90, 45, 180, 180, 90, 10).
m1 ... m6	Selecciona el motor a calibrar (M1 base, M2 hombro, M3 codo, M4 muñeca vertical, M5 muñeca rotativa, M6 pinza).
step N	Fija el incremento de los comandos +/- en N grados (p. ej. step 5).
+ / -	Suma o resta el paso actual al motor seleccionado.
+N / -N	Suma o resta N grados directamente.
N (o go N)	Envía el ángulo absoluto N al motor seleccionado.

Tabla 3 Comandos sketch de medición/calibración

Cada movimiento queda registrado con una línea de log autodescriptiva, y pose imprime el estado completo. El sketch puede probarse primero dentro del propio simulador (cargándolo en el editor y ejecutándolo, con la consola interactiva haciendo de monitor Serie), lo que permite familiarizarse con los comandos sin tocar el brazo real.

A continuación, puede ver un momento de la salida por consola serial del simulador ejecutando el sketch de medición/calibración:

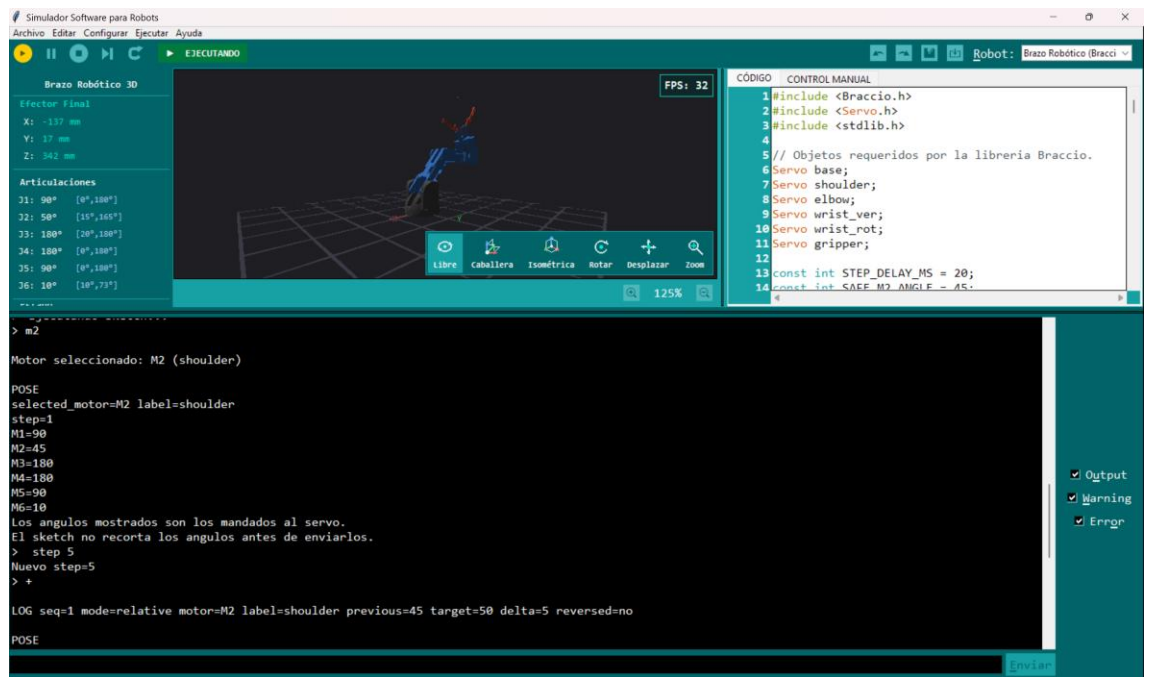


Figura 50 El sketch `braccio_medicion_real.ino` ejecutándose dentro del simulador

La consola interactiva del simulador reproduce la misma salida que el Monitor Serie del IDE de Arduino; la siguiente figura muestra cómo sería ejecutar el sketch en paralelo en ambos entornos para así llevar a cabo la calibración.

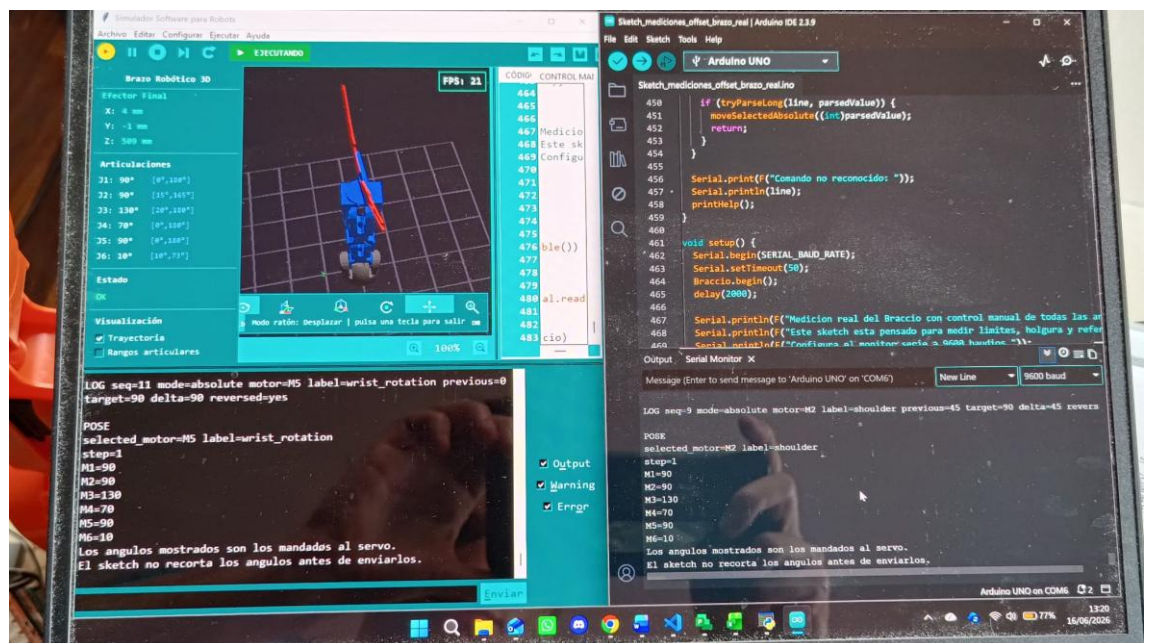


Figura 51 `braccio_medicion_real.ino` en paralelo: consola del simulador (izquierda) y Monitor Serie del IDE de Arduino (derecha) mostrando el mismo estado.

Procedimiento de medición, articulación por articulación

Orden recomendado: M1 (base), M5 (roll) y M6 (pinza) son las más cómodas; M2 (hombro) y M3 (codo) cargan el peso del brazo, conviene medirlas con el brazo recogido y sujetándolo ante cualquier movimiento inesperado. Nunca forzar un servo contra su tope mecánico:

1. Cargar `braccio_medicion_real.ino` en la placa con el IDE de Arduino (el fichero de codes/ puede pegarse en un sketch nuevo) y seleccionar la placa y puerto correspondientes.

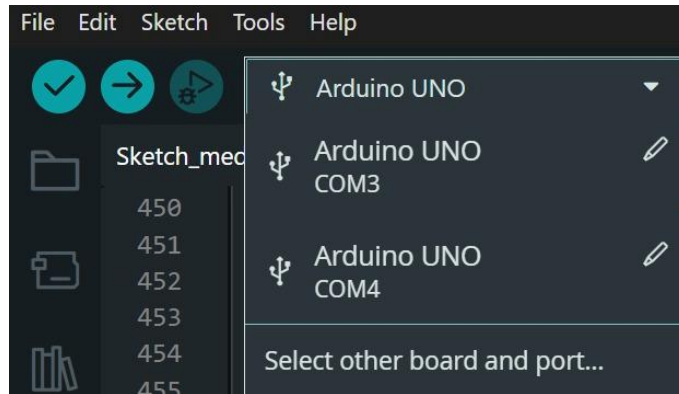


Figura 52 Selección de placa Arduino UNO y puerto en el IDE

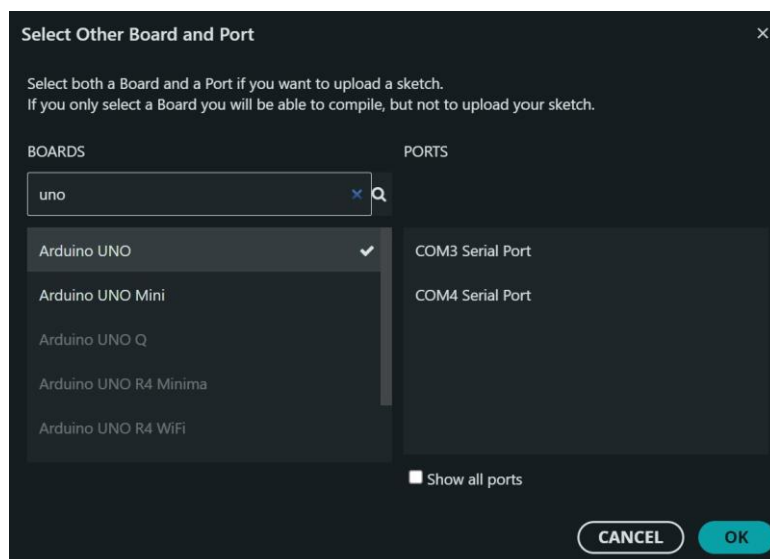


Figura 53 Opción «Select Other Board and Port» de la figura anterior

2. Abrir el Monitor Serie a 9600 baudios y arrancar el sketch que lleva el brazo a la pose base e imprime la ayuda como se ve en la Figura 51.

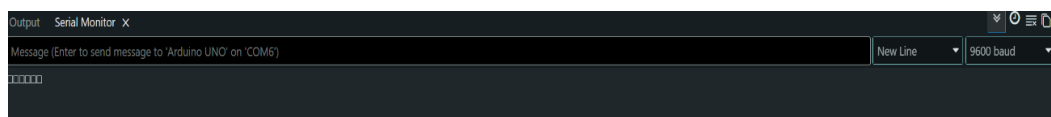


Figura 54 Monitor serie si detecta Arduino conectado en el puerto seleccionado

3. Seleccionar la articulación: m2 (por ejemplo). El sketch confirma el motor y su etiqueta.

4. Llevarla a su mínimo: con step 10 acercarse, cambiar a step 1 cerca del tope y avanzar hasta el último comando seguro (protocolo de límites del propio sketch). Anotar el valor digital alcanzado.
5. Medir el ángulo mecánico real con el transportador, alineado con el eje de giro de la articulación, y registrar el par (digital → real) en la plantilla.
6. Repetir los pasos 3–4 para la posición neutra (90, o 40 en la pinza) y para el máximo del rango.
7. Pasar a la siguiente articulación (reset si conviene volver a la pose segura) hasta completar las seis filas de la plantilla.

Volcado de las medidas a la tabla de calibración

Con la plantilla completa, editar el campo `servo_calibration` del preset `simulator/assets/presets/braccio_tinkerkit.json`: cada articulación recibe sus tres pares [valor_digital, ángulo_real_medido] como se mostró anteriormente en la Tabla 2 Plantilla de registro de calibración usada.

Los puntos no necesitan estar en los extremos exactos 0/90/180: el código interpola linealmente entre los puntos dados y satura fuera de ellos, así que lo importante es que los tres puntos cubran bien el recorrido útil.

Si se desea cambiar también el valor por defecto (el que se usa cuando un preset no define el campo), editar `DEFAULT_BRACCIO_SERVO_CALIBRATION` en la clase `Arm3DLayer` (`simulator/graphics/layers.py`).

Tras guardar el fichero, reiniciar el simulador. Internamente, `_to_model_value()` aplica la calibración (digital→real) y después la traslación canónica de -90° al actualizar el modelo, y `_to_control_value()` hace la interpolación inversa (real→digital) cuando el modelo actualiza el HUD y los deslizadores; no hay nada más que tocar.

Verificación de la calibración

1. Ejecutar el mismo sketch (`braccio_medicion_real.ino` u otro de movimientos fijos como, `Prueba_Servo_Library.ino`) simultáneamente en el simulador y en el Braccio físico.
2. Ordenar varias posturas representativas (por ejemplo, go 0, go 45, go 90, go 135 sobre dos o tres articulaciones) y comparar visualmente el brazo real con el simulado, midiendo con el transportador en caso de duda.
3. Criterio orientativo: si alguna articulación difiere en más de $\sim 3\text{--}5^\circ$ en parte del recorrido, revisar sus tres puntos (especialmente que el punto neutro se midiera con el brazo descargado) y repetir la medición de esa articulación.

La siguiente figura muestra cómo debería de quedar en la verificación final el Braccio físico y el modelo simulado adoptando la misma postura ordenada por el sketch (mismos valores para cada articulación impresos).

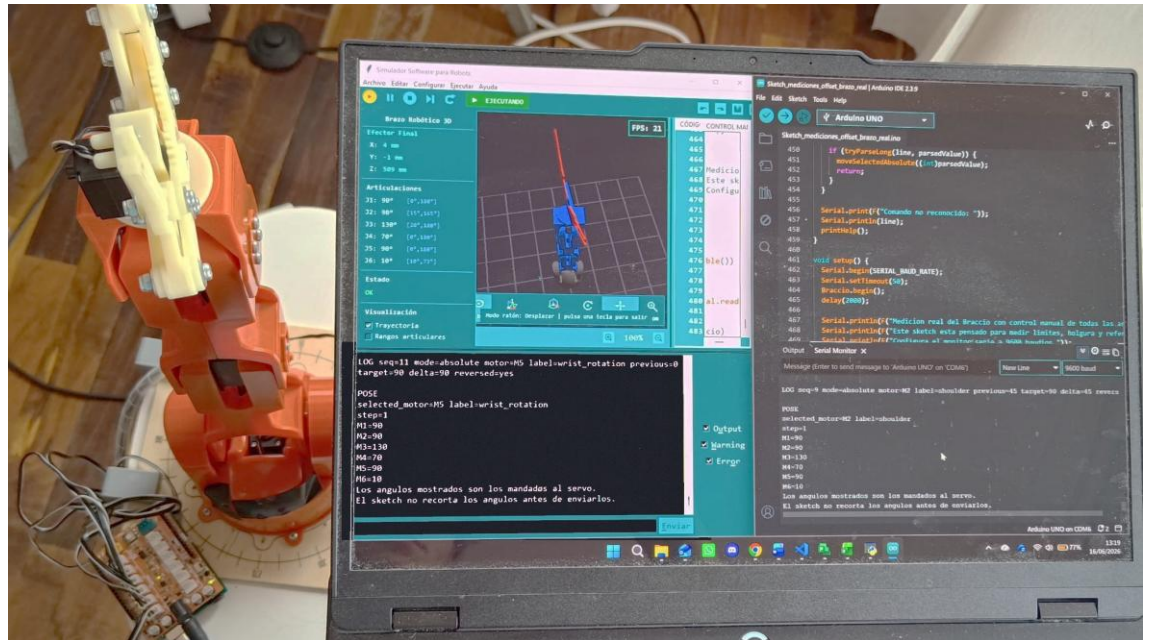


Figura 55 Comparación entre el Braccio físico y el modelo simulado en la misma postura durante la verificación de la calibración